

1. 课程名称：计算机体系结构
2. 课程类型：本科生必修课
3. 学生专业：计算机
4. 学时：48学时/12周
5. 讲课教师：夏宏
 - 办公室：E705, E1125
 - 邮 箱：summerday@ncepu.edu.cn

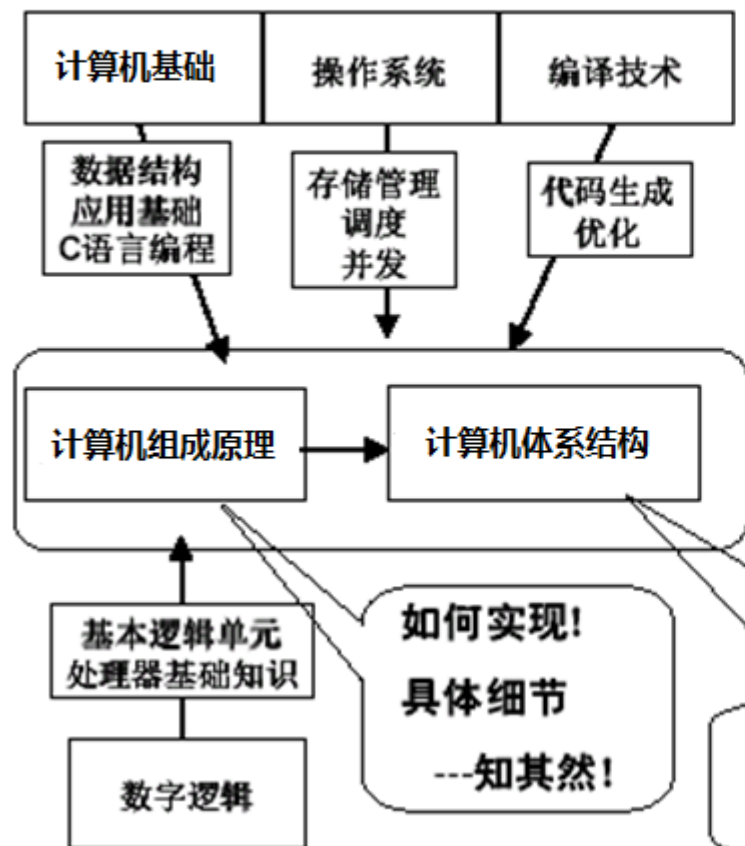
➤ 雨课堂：



课堂暗号：7KBAR

6. 预备知识：计算机原理，操作系统，编译原理，电路，汇编语言，C语言

本课程在教学安排中的地位



- 计算机应用需要什么?
- 操作系统需要那些功能支持?
- 优化编译可以利用和实现哪些功能?
- 我们能够建造什么样的机器?
- 今后的计算机将会怎样?

计算机系统结构研究人员
必须具有宽厚的专业知识!

1. 分析+评测—知其所以然!
2. 并行计算机系统结构引论

1. 教材:

计算机系统结构（第2版），张晨曦，高等教育出版社，2015

2. 主要参考书

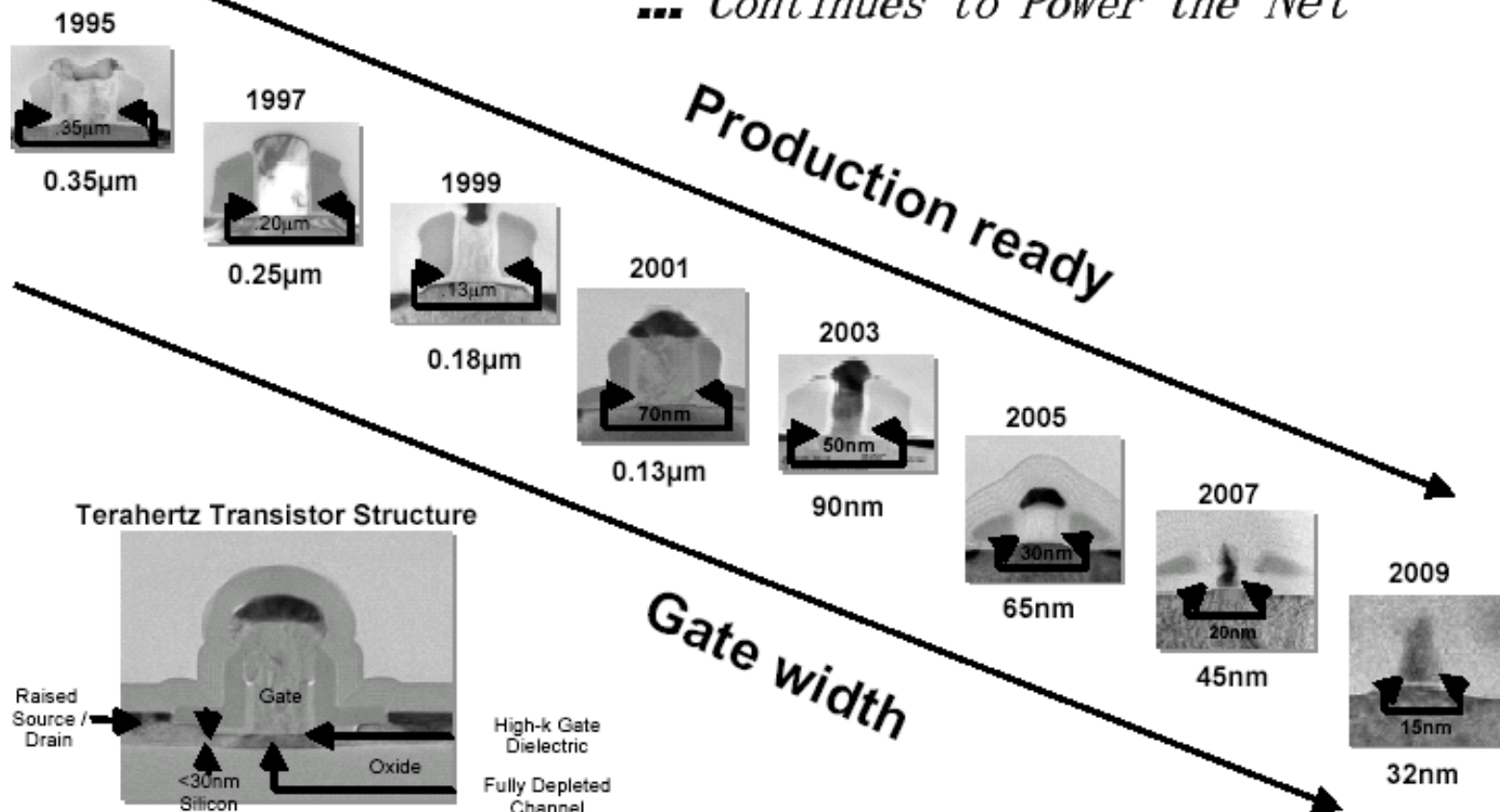
John L. Hennessy, David A. Patterson, “Computer Architecture:
A Quantitative Approach”, Fifth Edition, 2010

- 1.1 [引言](#)
- 1.2 [计算机系统结构的概念](#)
- 1.3 [定量分析技术](#)
- 1.4 [计算机系统结构的发展](#)
- 1.5 [计算机系统结构中并行性的发展](#)

1.1 引言

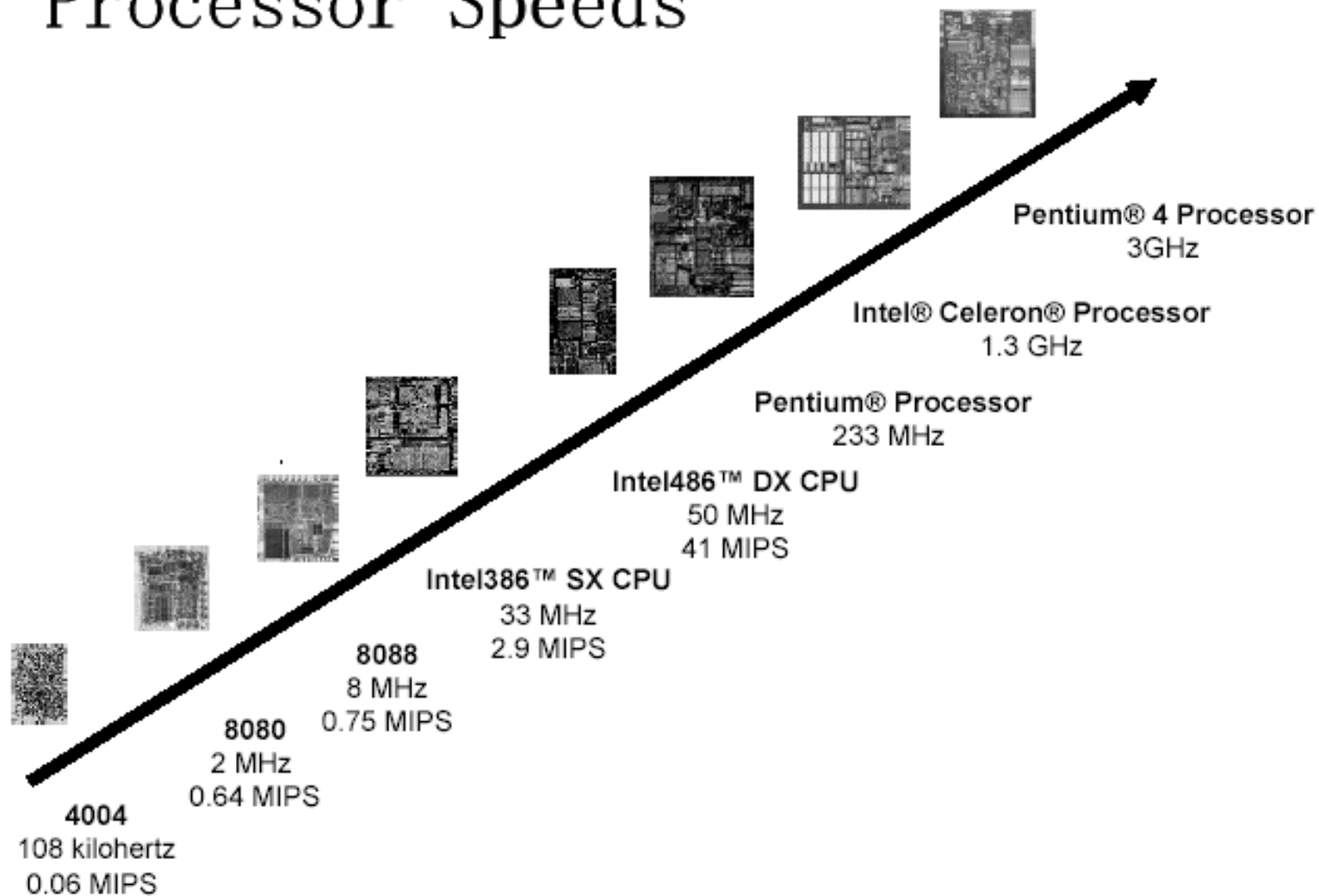
1. 第一台通用电子计算机诞生于1946年
2. 计算机技术的飞速发展得益于两个方面
 - 计算机制造技术的发展 稳定 摩尔定律
 - 计算机系统结构的创新 不稳定

Moore's Law

... Continues to Power the Net

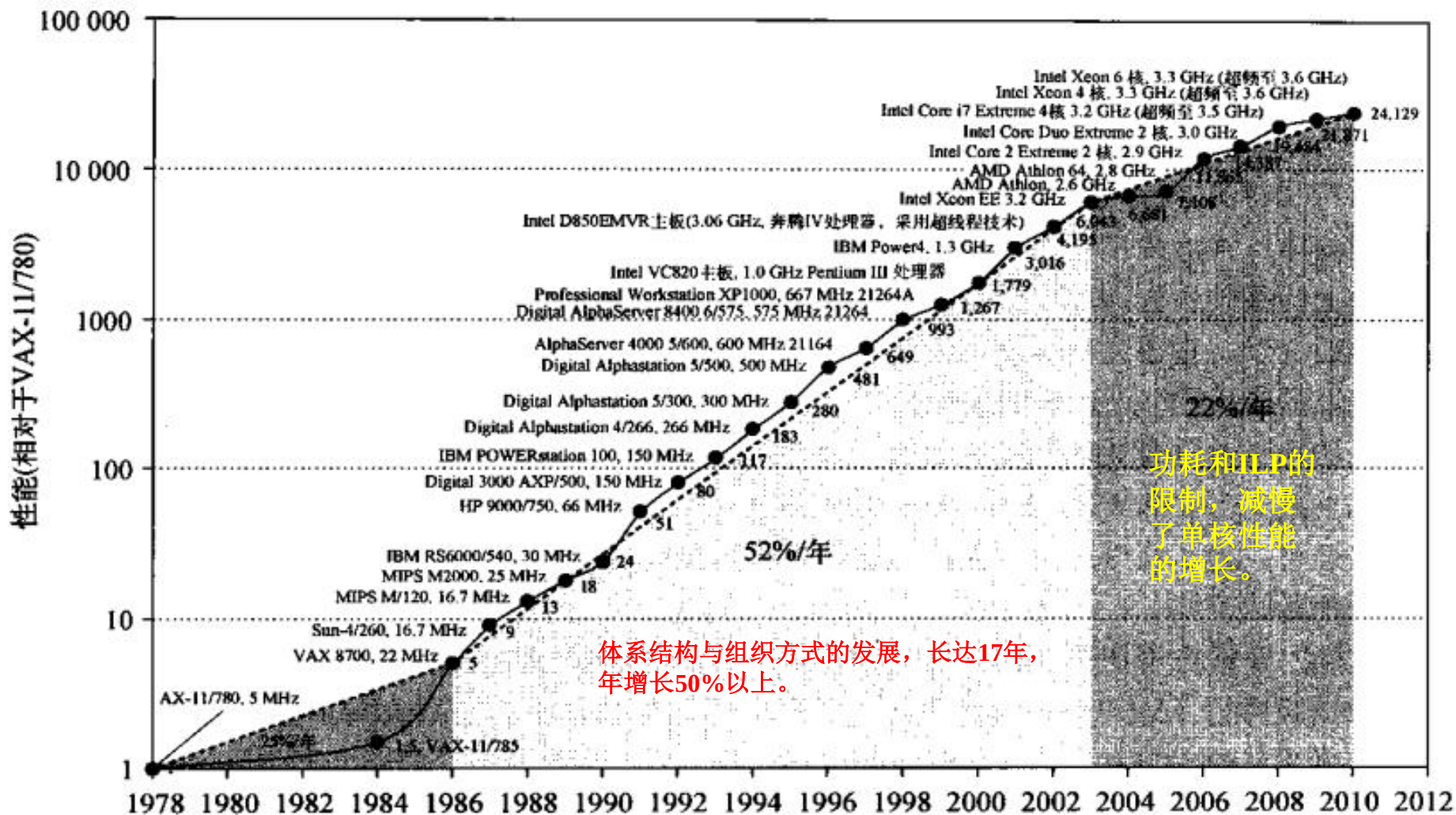
Source: Intel

Processor Speeds



3. 经历了4个发展过程

时 间	原 因	每年的性能增长
1946年起的25年	两种因素都起着主要的作用	25%
20世纪70年代末—80年代初	大规模集成电路和微处理器出现, 以集成电路为代表的制造技术的发展	约35%
20世纪80年代中开始	RISC结构的出现, 系统结构不断更新和变革, 制造技术不断发展	52%以上 维持了约17年
2002年以来	3个 (见9)	约22%



自20世纪70年代后期以来处理器性能的增长

- 功耗问题（已经很大）。
- 可以进一步有效开发的指令级并行性已经很少。
- 存储器访问速度的提高缓慢。

技术工艺发展趋势

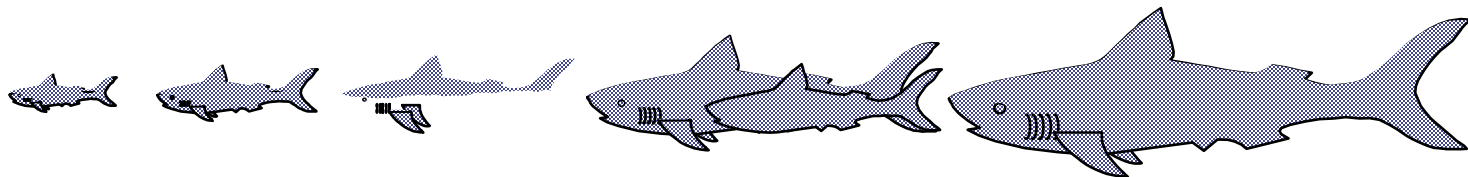
类型	容量	速度（反应时间）
Logic	每三年两倍	每三年两倍
DRAM	每三年四倍	每十年两倍
Disk	每三年四倍	每十年两倍

系统结构的重大转折：

从单纯依靠指令级并行转向开发线程级并行和数据级并行。

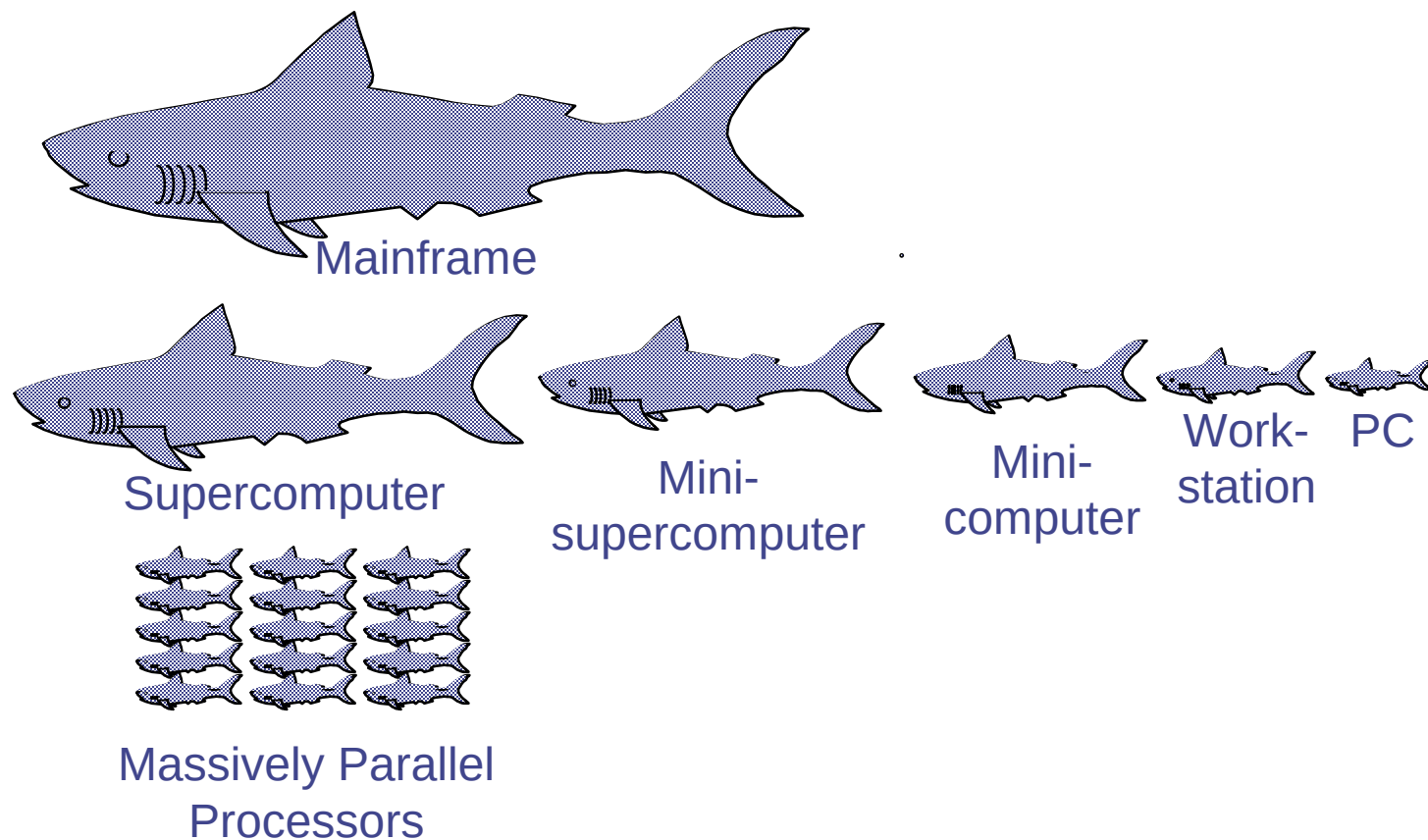
计算机系统结构在计算机的发展中有着极其重要的作用。

自然界中的生物链

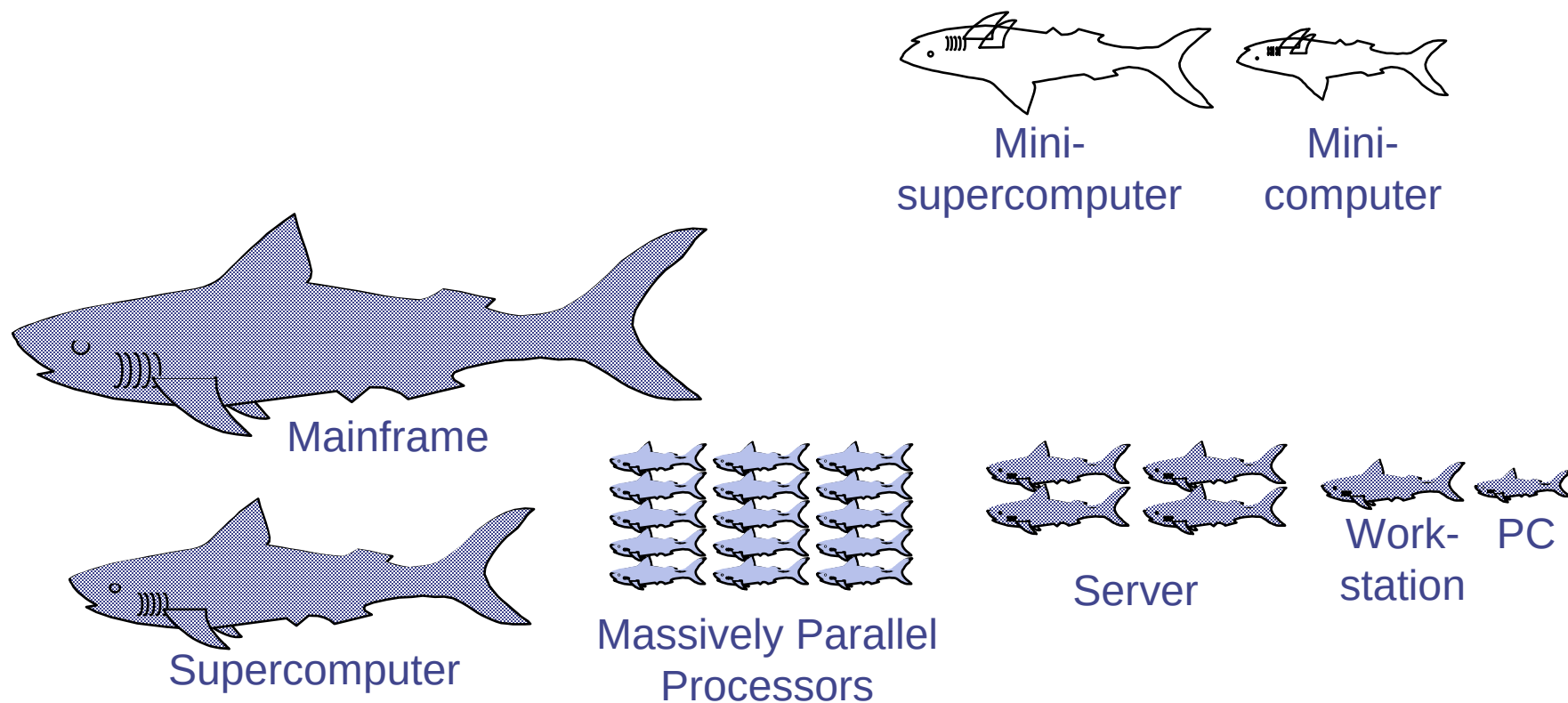


大鱼吃小鱼

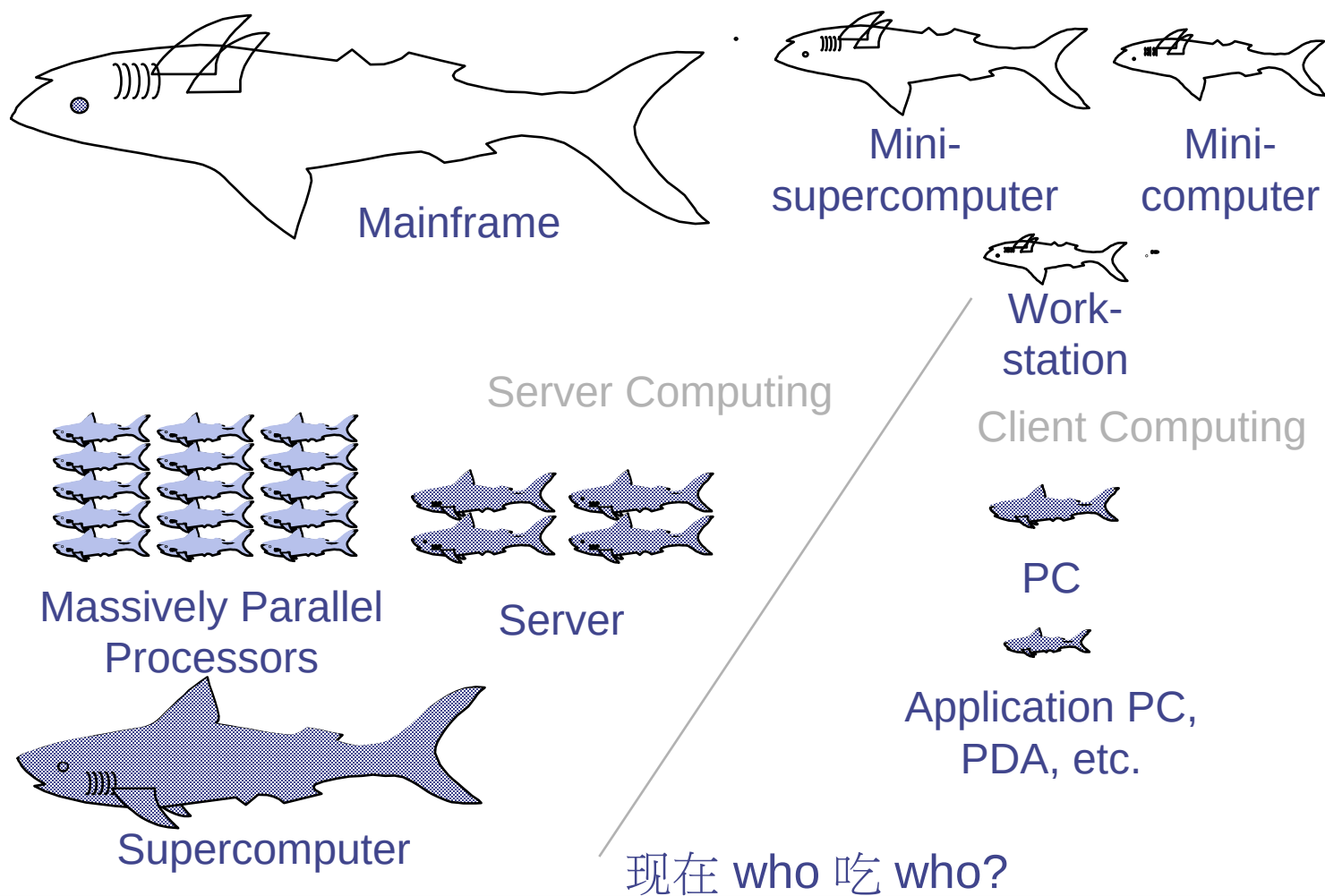
1988 年计算机的生物链



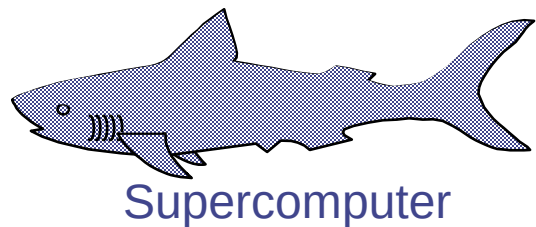
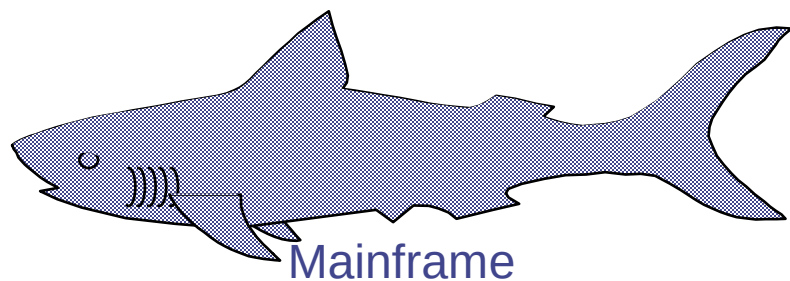
1998年计算机的生物链



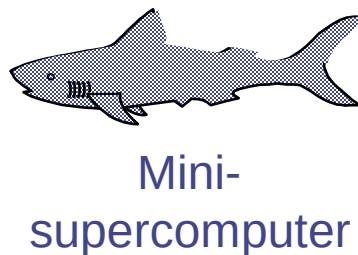
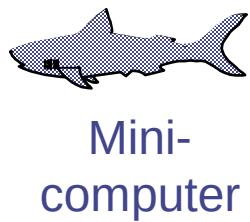
2000年计算机的生物链?



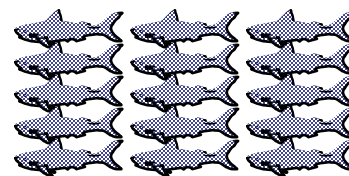
1988 年计算机工艺的食物链



ECL

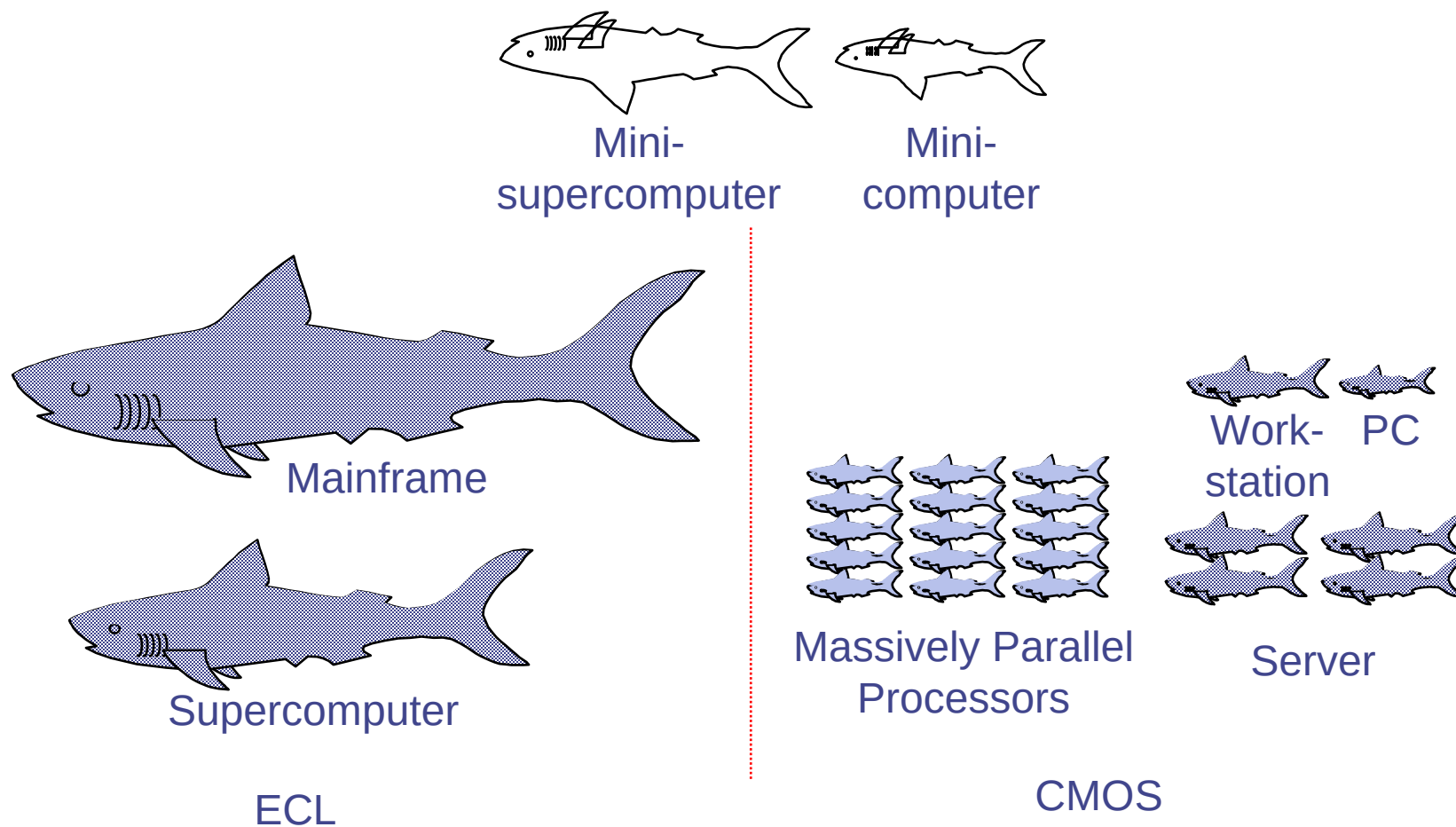


TTL

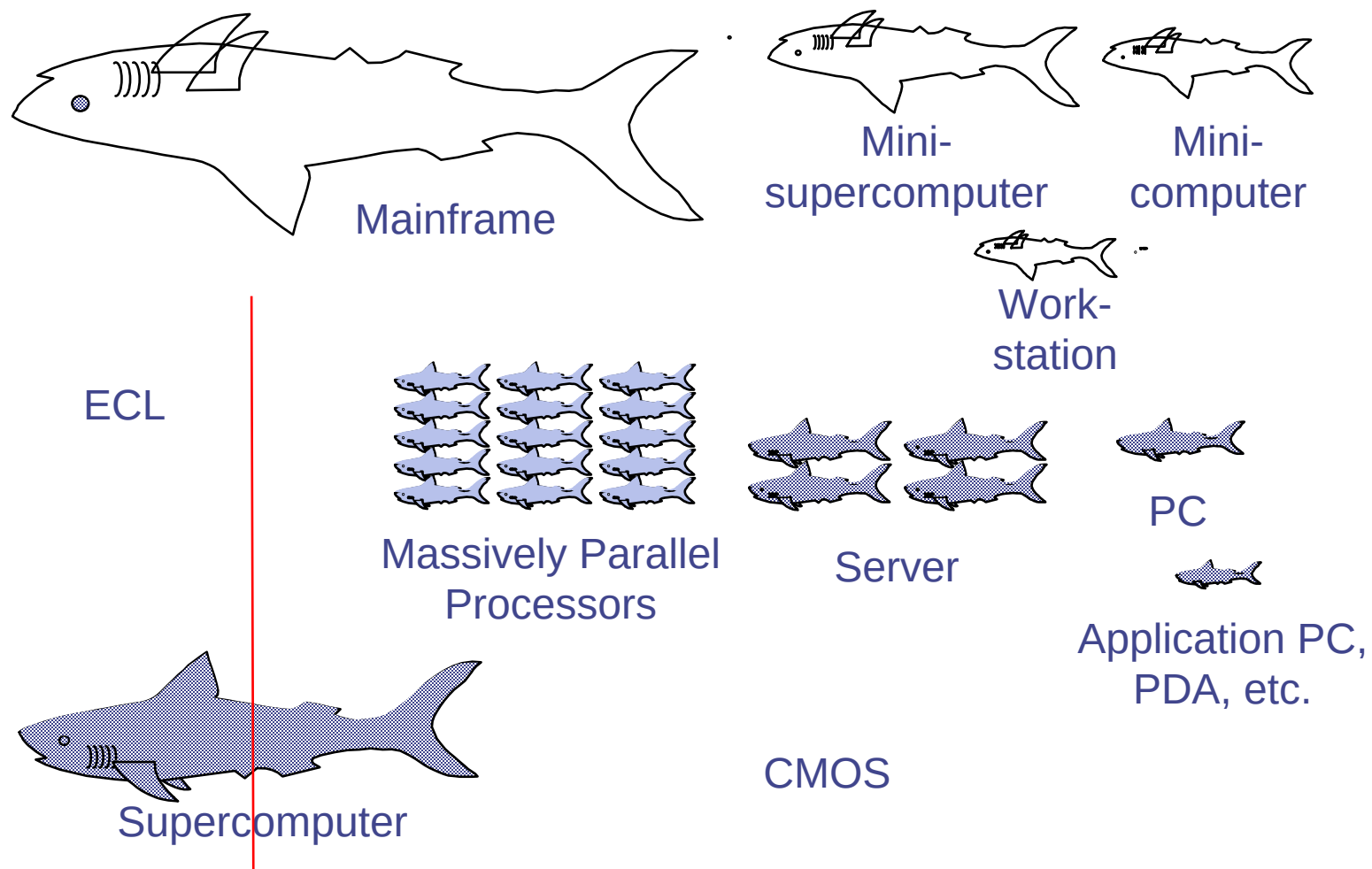


CMOS

1998年计算机工艺的食物链



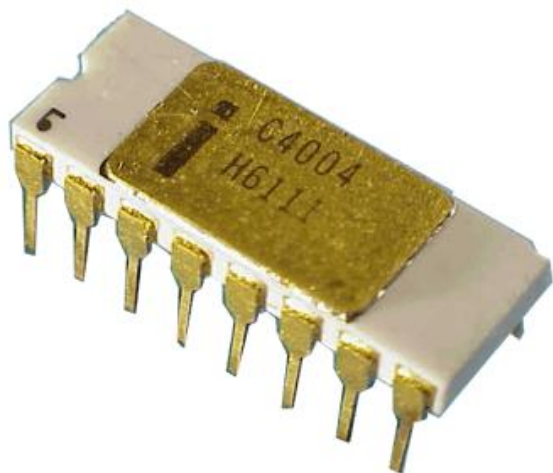
2000年计算机工艺的食物链?



微处理器的发展

Intel 4004

1971年 Intel公司推出了世界上第一款微处理器4004，这是第一个可用于微型计算机的四位微处理器，它包含2.3K个晶体管， 12mm^2 。



几乎同时，TI推出了 TMC1795
3.2K transistors 30mm^2

Intel 8008

1972年 Intel公司推出了8008微处理器，这是第一个可用于微型计算机的八位微处理器，它包含3.5K个晶体管， 20mm^2 。



Intel 8080 3000 4040

1974年 Intel公司推出了3款微处理器。

Motorola 6800

1976年

Intel
8085

Zilog
Z80

1978年

Intel
8086 (P1) X86系列 (CISC)

1979年

Intel
8088

Zilog
Z8000

1980年

Intel

8031, 51

1982年

Intel

80286 (P2) MMU

1984年

MIPS (RISC)

R2000

1985年

Intel

80386DX (P3)

ARM

ARMv1

1987年

SPARC

SPARC V7

1989年

Intel

80486DX (P4)

Pipelined

1993年

Superscalar

Pentium(80501) (P5) AMD K5 AMDK6

1995年

RISC core, L2 cache, superpipelining

Pentium Pro Pentium II AMDK6-2 (P6)

1997年

MMX

Pentium MMX

1998年

power

Pentium II Mobile

1999年

Pentium III , StrongARM

2000年

Hyper-Threading

Pentium 4 Athlon

2001年

IA-32

Itanium

2006年

dual-core

Intel Core

2007年

IA-64,EPIC

Athlon 64 Core 2

我国高性能计算机

银河系列：国防科技大学

曙光系列：中国科学院计算所

神威系列：国家并行计算机工程研究中心

1.2 计算机系统结构的概念

1.2.1 计算机系统的层次结构

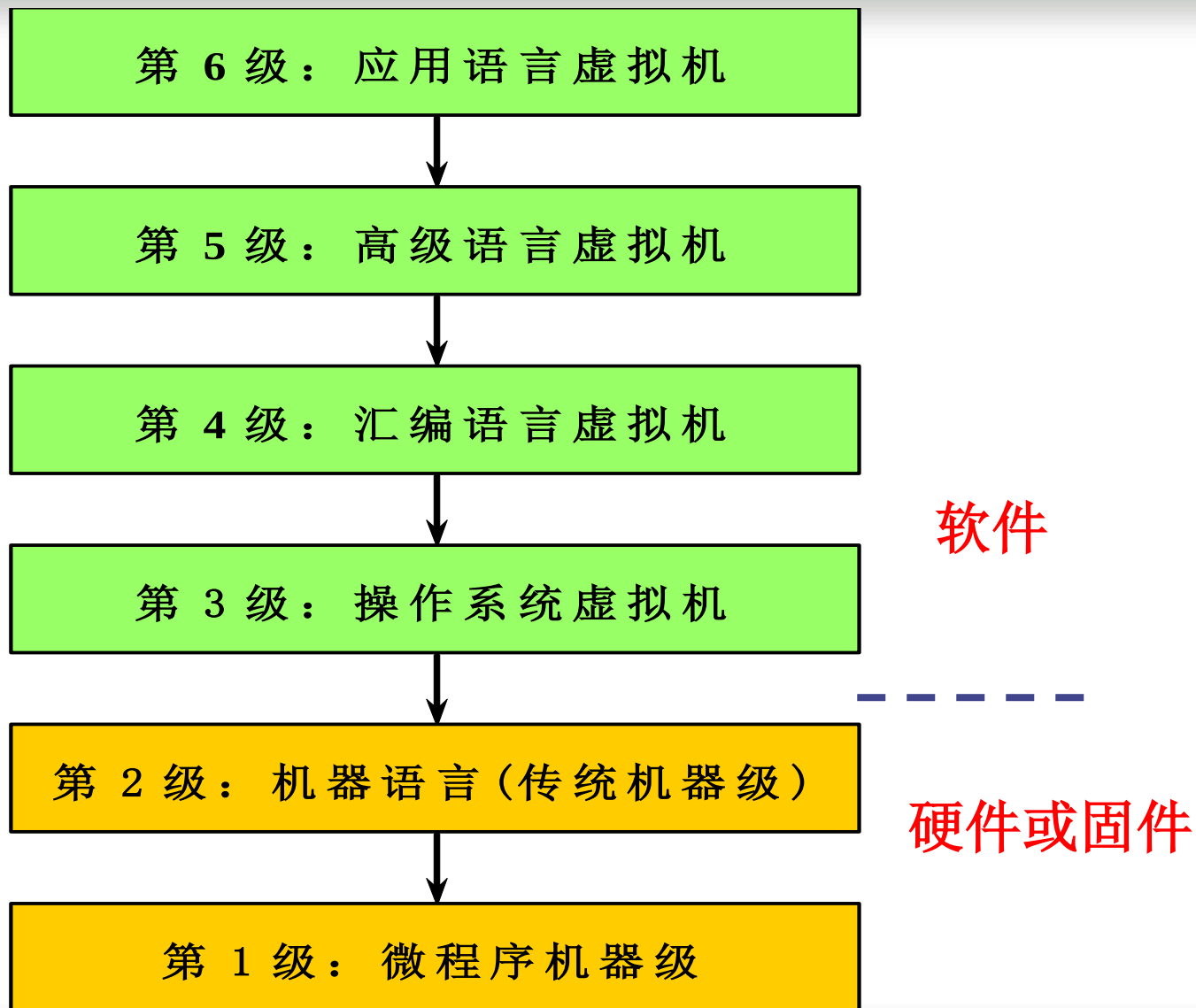
1. 计算机系统=硬件/固件+软件

2. 计算机语言从低级向高级发展

高一级语言的语句相对于低一级语言来说功能更强，更便于应用，但又都以低级语言为基础。

3. 从计算机语言的角度，把计算机系统按功能划分成多级层次结构。

➤ 每一层以一种语言为特征



分层的意义：有利于正确理解计算机系统的工作。

- ① 有利于正确理解计算机系统的工作，明确软件、硬件和固件在计算机系统中的地位和作用。
- ② 有利于理解各种语言的实质及其实现。
- ③ 有利于探索虚拟机器新的实现方法，设计新的计算机系统。

- **虚拟机**：由软件实现的机器(*vmware*)
- 语言实现的两种基本技术
 - **翻译**：先把 $N+1$ 级程序全部转换成 N 级程序后，再去执行新产生的 N 级程序，在执行过程中 $N+1$ 级程序不再被访问。
 - **解释**：每当一条 $N+1$ 级指令被译码后，就直接去执行一串等效的 N 级指令，然后再去取下一条 $N+1$ 级的指令，依此重复进行。

解释执行比编译后再执行所花的时间多，但占用的存储空间较少。

1.2.2 计算机系统结构的定义

1. 计算机系统结构的经典定义

Amdahl在1964年提出 “Computer Architecture”

程序员(机器语言或编译程序设计者)所看到的计算机属性，即概念性结构与功能特性。计算机系统结构的定义随时代而变化！

2. 按照计算机系统的多级层次结构，不同级程序员所看到的计算机具有不同的属性。

3. 透明性

在计算机技术中，把这种本来存在的事物或属性，但从某种角度看又好像不存在的概念称为透明性。

4. Amdahl提出的系统结构

传统机器语言级程序员所看到的计算机属性。

5. 广义的系统结构定义：指令集结构、组成、硬件 (计算机设计的3个方面)

6. 对于通用寄存器型机器来说，这些属性主要是指：

➤ 指令系统

包括机器指令的操作类型和格式、指令间的排序和控制机构等。

➤ 数据表示

硬件能直接识别和处理的数据类型。

➤ 寻址规则

包括最小寻址单元、寻址方式及其表示。

- 寄存器定义
(包括各种寄存器的定义、数量和使用方式)
- 中断系统
(中断的类型和中断响应硬件的功能等)
- 机器工作状态的定义和切换
(如管态和目态等)
- 存储系统
(主存容量、程序员可用的最大存储容量等)
- 信息保护
(包括信息保护方式和硬件对信息保护的支持)

➤ I/O结构

包括I/O连接方式、处理机/存储器与I/O设备之间数据传送的方式和格式以及I/O操作的状态等

计算机系统结构概念的实质：

确定计算机系统中软、硬件的界面，界面之上是软件实现的功能，界面之下是硬件和固件实现的功能。

Computer Architecture Changing Definition

- 1950年到 1960年: Computer Architecture Course

Computer Arithmetic (计算机算法)

- 1970年到1980年中期: Computer Architecture Course

Instruction Set Design, especially ISA appropriate for compilers

指令集设计, 尤其是对编译器优化的ISA

- 90年代: Computer Architecture Course

Design of CPU, memory system, I/O system, Multiprocessors

CPU的设计, 存储系统, I/O系统, 多处理器

- 2000年: Instruction Level Parallelism (IA-64) (指令级并行)

- 2010年: Cluster、MPP

1.2.3 计算机组成和计算机实现

1. 计算机系统结构：计算机系统的软、硬件的界面

即机器语言程序员所看到的传统机器级所具有的属性。

2. 计算机组成：计算机系统结构的逻辑实现

- 包含物理机器级中的数据流和控制流的组成以及逻辑设计等。
- 着眼于：物理机器级内各事件的排序方式与控制方式、各部件的功能以及各部件之间的联系。

3. 计算机实现：计算机组成的物理实现

- 包括处理机、主存等部件的物理结构，器件的集成度和速度，模块、插件、底板的划分与连接，信号传输，电源、冷却及整机装配技术等。
- 着眼于：器件技术（起主导作用）、微组装技术。

一种体系结构可以有多种组成。

一种组成可以有多种物理实现。

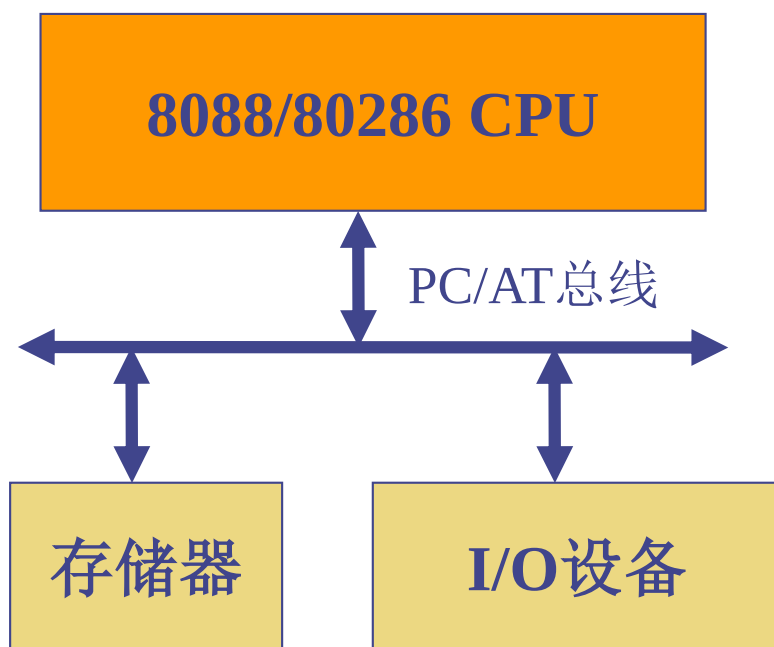
4. 系列机

由同一厂家生产的具有相同系统结构、但具有不同组成和实现的一系列不同型号的计算机。

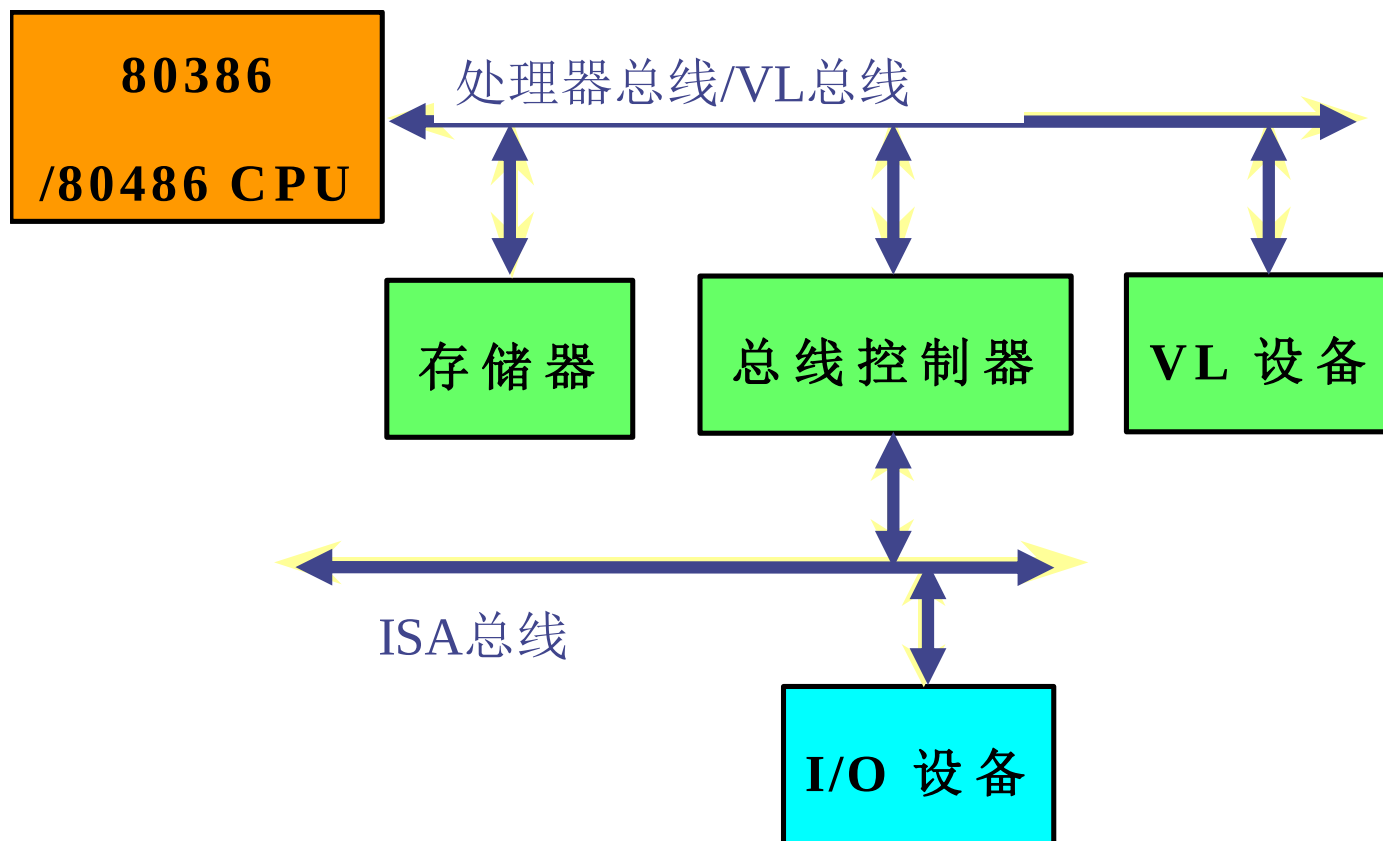
例如，IBM公司的IBM 370系列，Intel公司的x86系列等。

IBM PC系列机

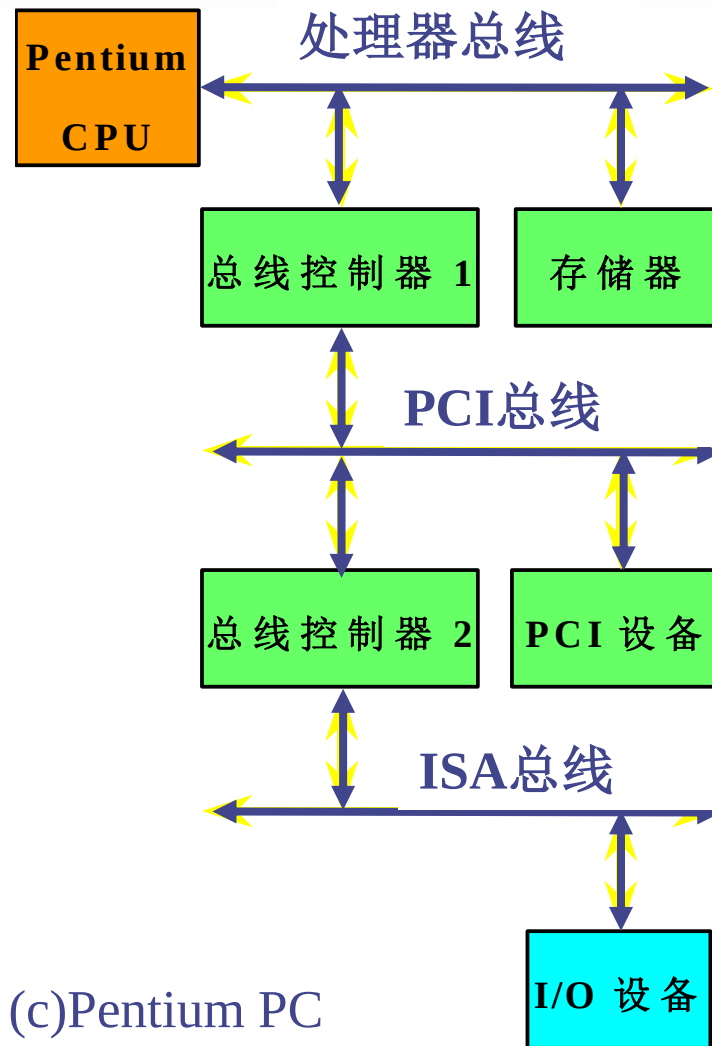
处理器、处理器字宽、主要I/O总线、存储空间、主要操作系统和计算机结构



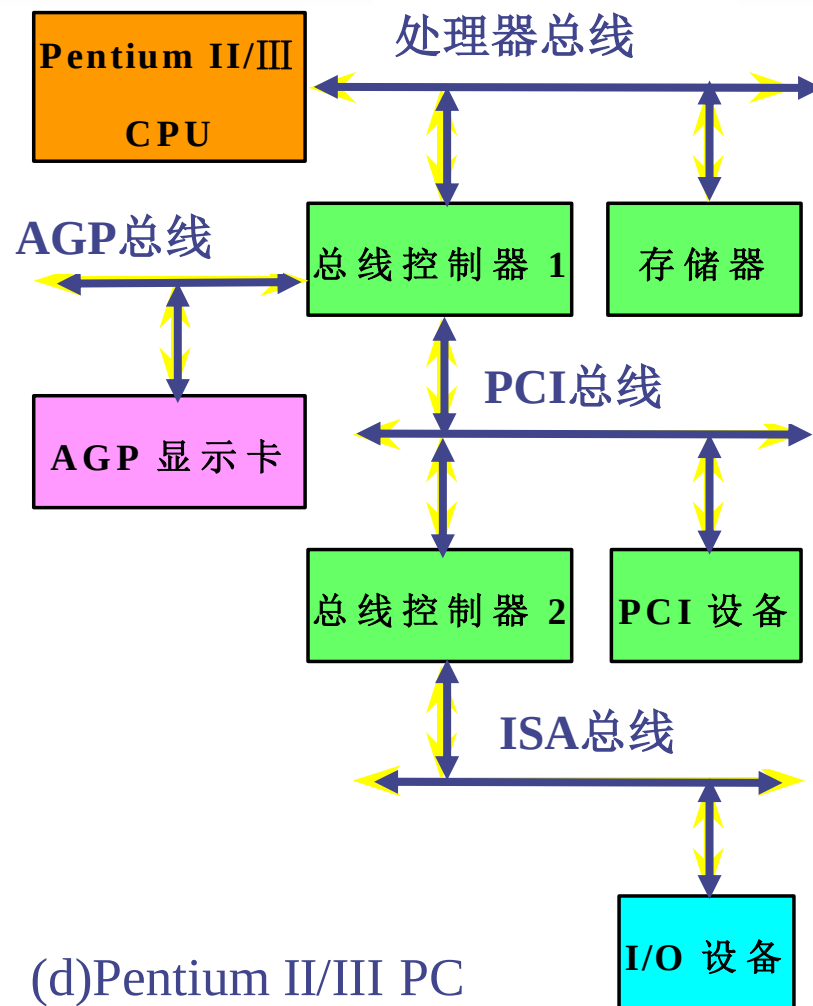
(a)PC XT和PC AT

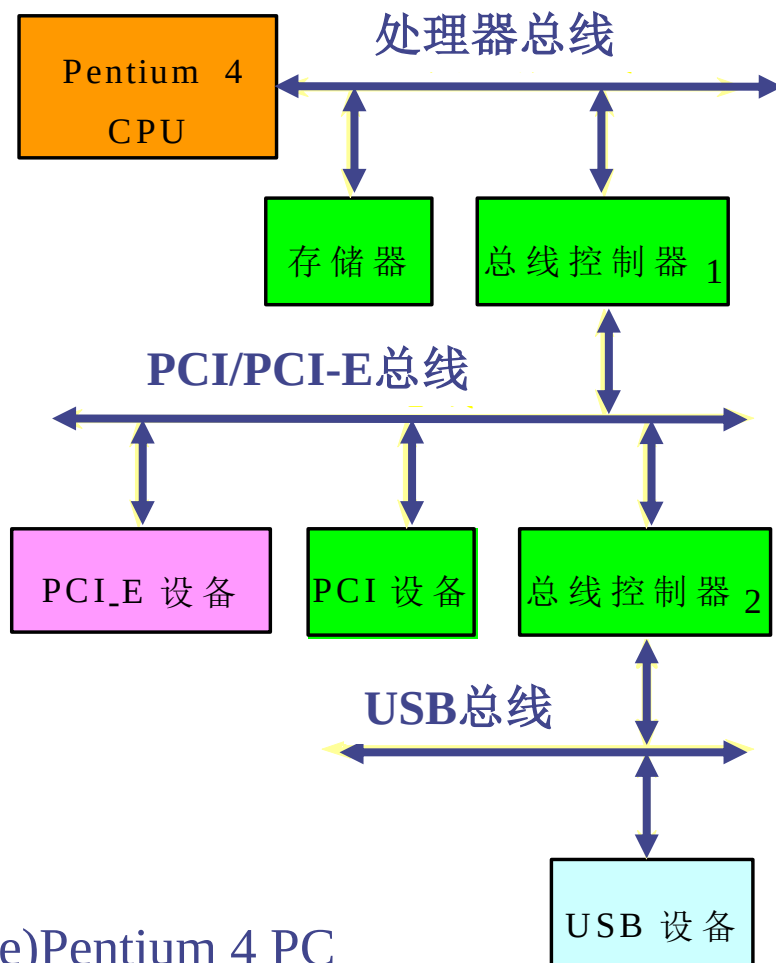


(b)80386/80486 PC



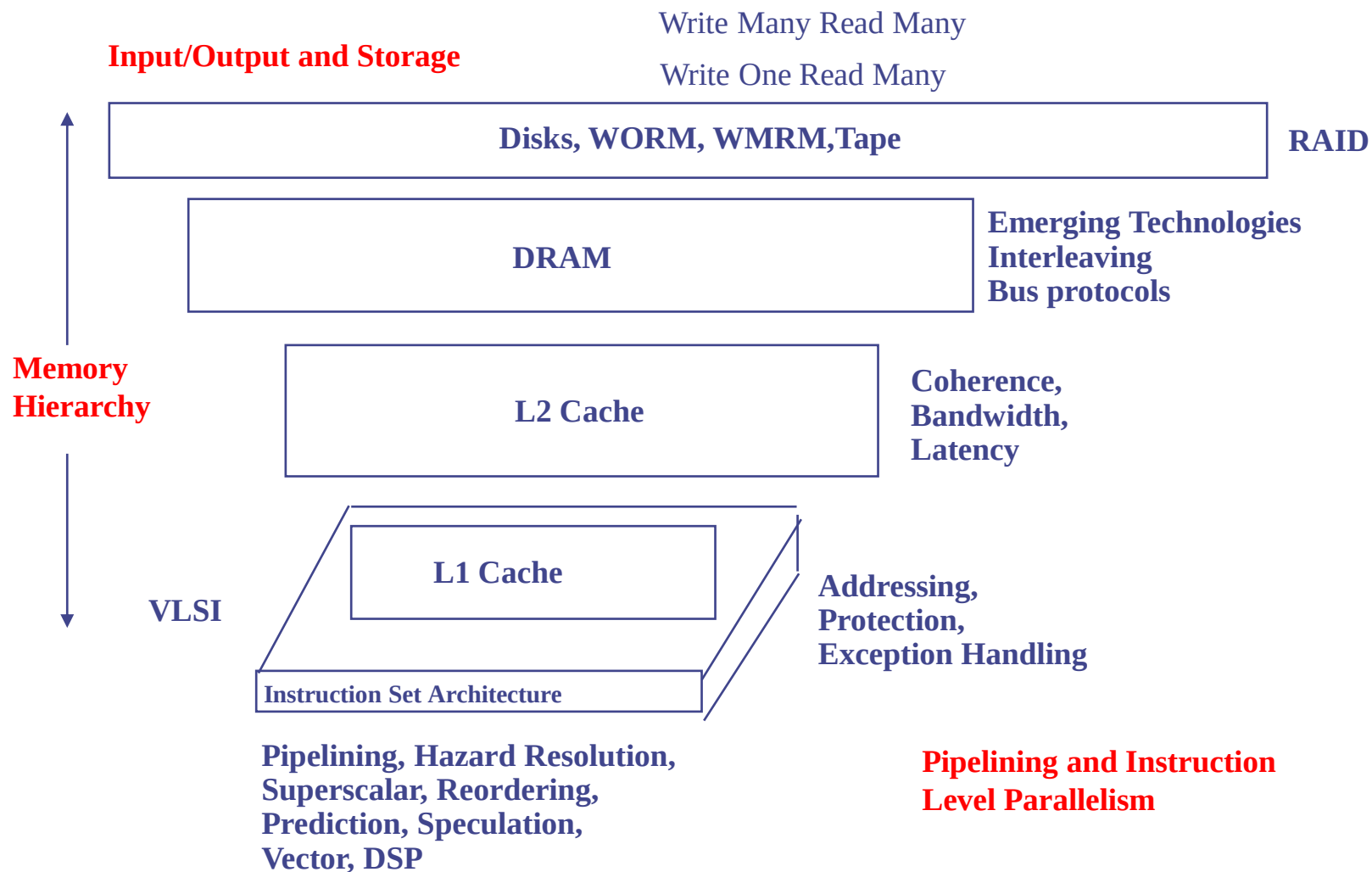
(c)Pentium PC





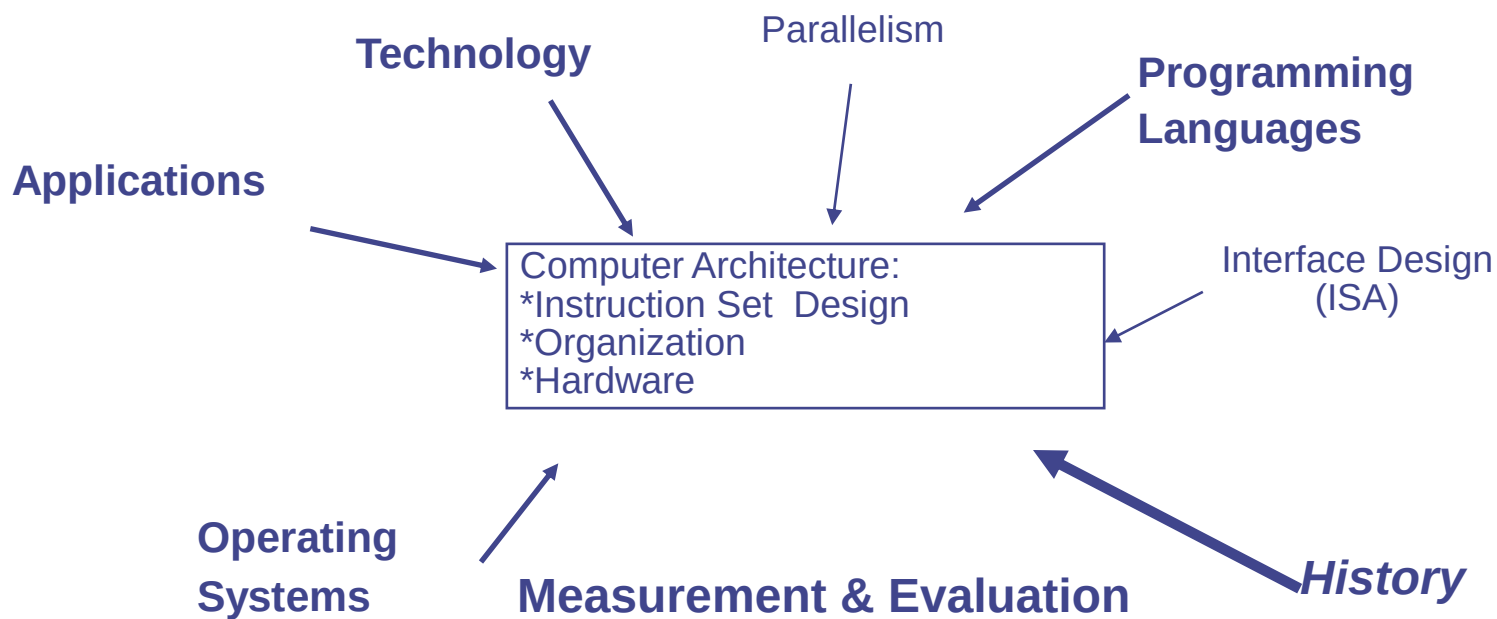
(e)Pentium 4 PC

Computer Architecture Topics



Computer Course Focus

Understanding the design techniques, machine structures, technology factors, evaluation methods that will determine the form of computers in 21st Century



1.2.4 计算机系统结构的分类

常见的计算机系统结构分类法有两种：

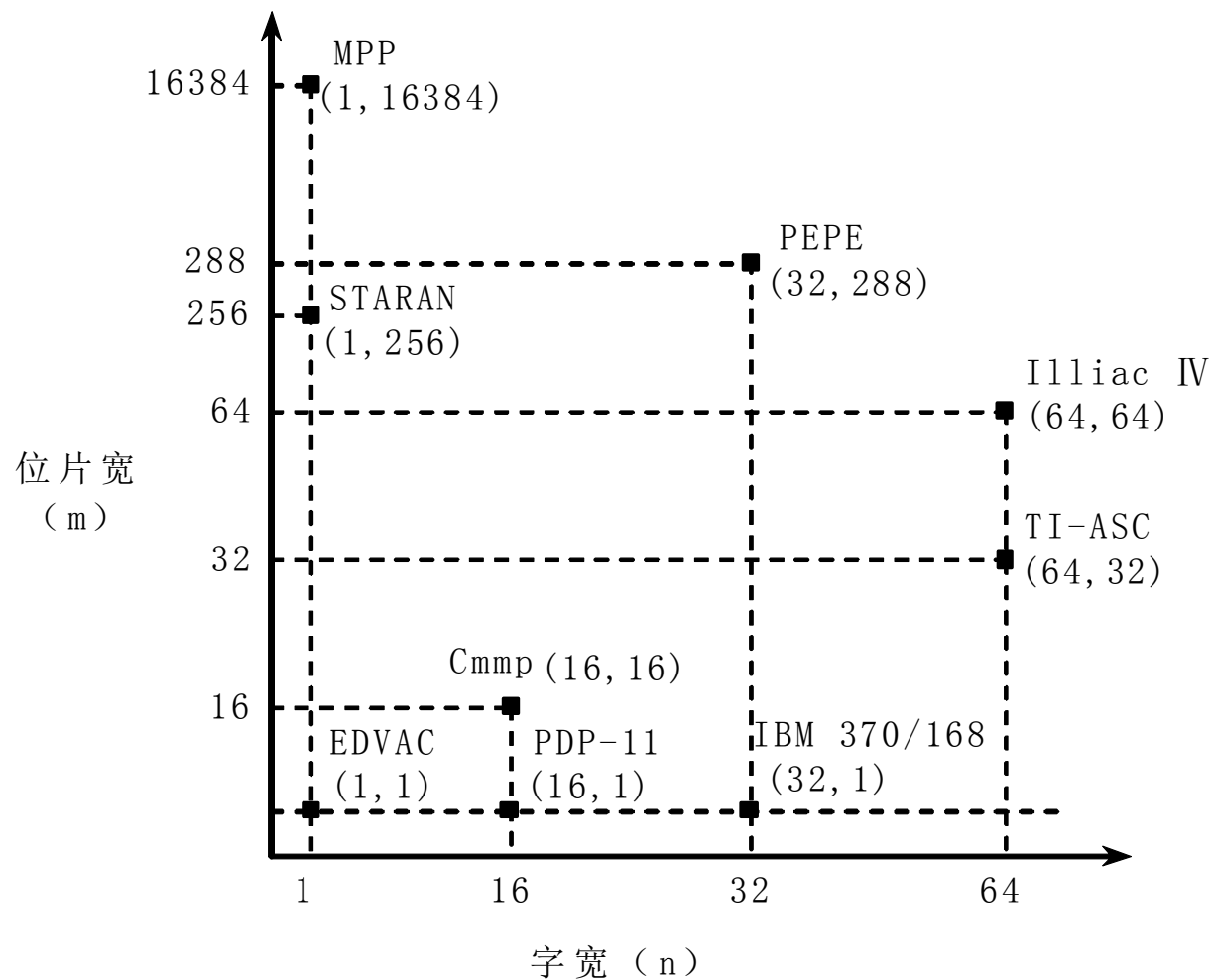
Flynn分类法、冯氏分类法

1. 冯氏分类法

- 用系统的最大并行度对计算机进行分类。
- **最大并行度**：计算机系统在单位时间内能够处理的最大的二进制位数。

用平面直角坐标系中的一个点代表一个计算机系统，其横坐标表示字宽（ n 位），纵坐标表示一次能同时处理的字数（ m 字）。 $m \times n$ 就表示了其最大并行度。

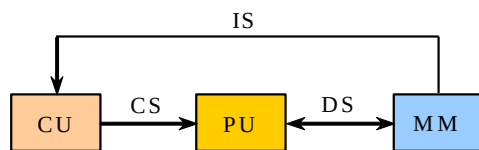
- 字串位串：WSBS
- 字并位串：WPBS
- 字串位并：WSBP
- 字并位并：WPBP



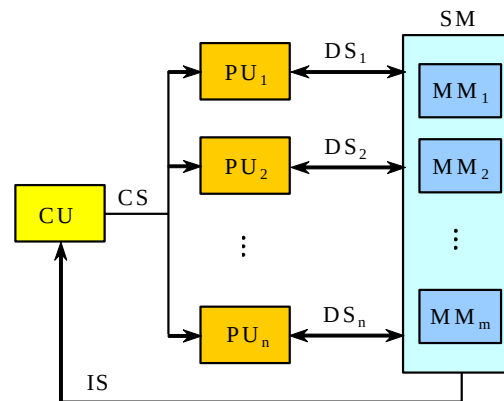
2. Flynn分类法

- 按照指令流和数据流的多倍性进行分类。
- **指令流**：计算机执行的指令序列。

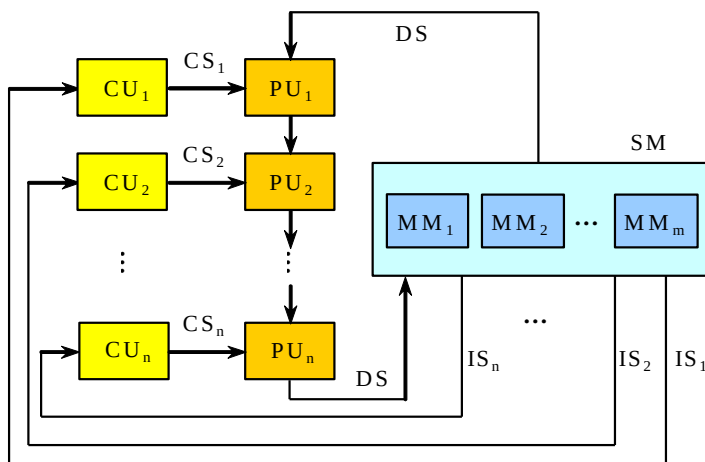
- **数据流**：由指令流调用的数据序列。
- **多倍性**：在系统受限的部件上，同时处于同一执行阶段的指令或数据的最大数目。
- Flynn分类法把计算机系统的结构分为4类：
 - ❑ 单指令流单数据流(SISD)
 - ❑ 单指令流多数据流(SIMD)
 - ❑ 多指令流单数据流(MISD)
 - ❑ 多指令流多数据流(MIMD)
- 4类计算机的基本结构
 - IS：指令流，DS：数据流，CS：控制流，
 - CU：控制部件，PU：处理部件，MM和SM：存储器。



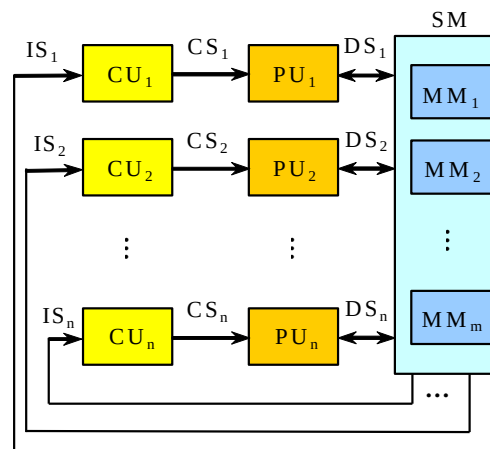
(a) SISD 计算机



(b) SIMD 计算机



(c) MISD 计算机



(d) MIMD 计算机

1.3 定量分析技术

1.3.1 计算机系统设计的定量原理

4个定量原理：

1. 以经常性事件为重点

- 对经常发生的情况采用优化方法的原则进行选择，以得到更多的总体上的改进。
- 优化是指分配更多的资源、达到更高的性能或者分配更多的电能等。

2. Amdahl 定律

加快某部件执行速度所能获得的系统性能加速比，受限于该部件的执行时间占系统中总执行时间的百分比。

系统性能加速比：

$$\text{加速比} = \frac{\text{系统性能}_{\text{改进后}}}{\text{系统性能}_{\text{改进前}}} = \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}}$$

➤ 加速比依赖于两个因素

- **可改进比例：**在改进前的系统中，可改进部分的执行时间在总的执行时间中所占的比例。

它总是小于等于1。

例如：一个需运行60秒的程序中有20秒的运算可以加速，那么这个比例就是20/60。

- **部件加速比：**可改进部分改进以后性能提高的倍数。它是改进前所需的执行时间与改进后执行时间的比。一般情况下部件加速比是大于1的。

例如：若系统改进后，可改进部分的执行时间是2秒，而改进前其执行时间为5秒，则部件加速比为5/2。

➤ 改进后程序的总执行时间

总执行时间_{改进后} = 不可改进部分的执行时间 +
可改进部分改进后的执行时间

$$\begin{aligned}\text{总执行时间}_{\text{改进后}} &= (1 - \text{可改进比例}) \times \text{总执行时间}_{\text{改进前}} \\ &\quad + \frac{\text{可改进比例} \times \text{总执行时间}_{\text{改进前}}}{\text{部件加速比}} \\ &= \left[(1 - \text{可改进比例}) + \frac{\text{可改进比例}}{\text{部件加速比}} \right] \times \text{总执行时间}_{\text{改进前}}\end{aligned}$$

系统加速比为改进前与改进后总执行时间之比

$$\begin{aligned}\text{加速比} &= \frac{\text{总执行时间}_{\text{改进前}}}{\text{总执行时间}_{\text{改进后}}} \\ &= \frac{1}{(1 - \text{可改进比例}) + \frac{\text{可改进比例}}{\text{部件加速比}}}\end{aligned}$$

例1.1 将计算机系统中某一功能的处理速度提高到原来的20倍，但该功能的处理时间仅占整个系统运行时间的40%，则采用此提高性能的方法后，能使整个系统的性能提高多少？

解 由题可知，可改进比例 = 40% = 0.4，

部件加速比 = 20

根据Amdahl定律可知：

$$\text{总加速比} = \frac{1}{0.6 + \frac{0.4}{20}} = 1.613$$

采用此提高性能的方法后，能使整个系统的性能提高到原来的1.613倍。

例1.2 某计算机系统采用浮点运算部件后，使浮点运算速度提高到原来的20倍，而系统运行某一程序的整体性能提高到原来的5倍，试计算该程序中浮点操作所占的比例。

解 由题可知，部件加速比 = 20，系统加速比 = 5

根据Amdahl定律可知

$$5 = \frac{1}{(1 - \text{可改进比例}) + \frac{\text{可改进比例}}{20}}$$

由此可得：可改进比例 = 84.2%

即程序中浮点操作所占的比例为84.2%。

- Amdahl 定律：一种性能改进的递减规则
 - 如果仅仅对计算任务中的一部分做性能改进，则改进得越多，所得到的总体性能的提升就越有限。
- 重要推论：如果只针对整个任务的一部分进行改进和优化，那么所获得的加速比不超过
$$1 / (1 - \text{可改进比例})$$

3. CPU性能公式

- 执行一个程序所需的CPU时间

CPU时间 = 执行程序所需的时钟周期数 × 时钟周期时间

其中，时钟周期时间是系统时钟频率的倒数。

- 每条指令执行的平均时钟周期数CPI

(Cycles Per Instruction)

$CPI = \text{执行程序所需的时钟周期数} / IC$

IC: 所执行的指令条数

- 程序执行的CPU时间可以写成

$CPU\text{时间} = IC \times CPI \times \text{时钟周期时间}$

➤ CPU的性能取决于3个参数

- **时钟周期时间**：取决于硬件实现技术和计算机组成。
- **CPI**：取决于计算机组成和指令集结构。
- **IC**：取决于指令集结构和编译技术。

➤ 对CPU性能公式进行进一步细化

假设：计算机系统有 n 种指令；

CPI_i ：第 i 种指令的处理时间；

IC_i ：在程序中第 i 种指令出现的次数；

则

$$\text{CPU时钟周期数} = \sum_{i=1}^n (CPI_i \times IC_i)$$

$$\begin{aligned}\text{CPU时间} &= \text{执行程序所需的时钟周期数} \times \text{时钟周期时间} \\ &= \sum_{i=1}^n (\text{CPI}_i \times \text{IC}_i) \times \text{时钟周期时间}\end{aligned}$$

CPI 可以表示为

$$\text{CPI} = \frac{\text{时钟周期数}}{\text{IC}} = \frac{\sum_{i=1}^n (\text{CPI}_i \times \text{IC}_i)}{\text{IC}} = \sum_{i=1}^n (\text{CPI}_i \times \frac{\text{IC}_i}{\text{IC}})$$

其中, (IC_i/IC) 反映了第 i 种指令在程序中所占的比例。

例1.3 考虑条件分支指令的两种不同设计方法：

(1) CPU_A ：通过比较指令设置条件码，然后测试条件码进行分支。

(2) CPU_B ：在分支指令中包括比较过程。

在这两种CPU中，条件分支指令都占用2个时钟周期，而所有其他指令占用1个时钟周期。对于 CPU_A ，执行的指令中分支指令占20%；由于每条分支指令之前都需要有比较指令，因此比较指令也占20%。由于 CPU_A 在分支时不需要比较，因此 CPU_B 的时钟周期时间是 CPU_A 的1.25倍。问：哪一个CPU更快？如果 CPU_B 的时钟周期时间只是 CPU_A 的1.1倍，哪一个CPU更快呢？

解 我们不考虑所有系统问题，所以可用CPU性能公式。占用2个时钟周期的分支指令占总指令的20%，剩下的指令占用1个时钟周期。所以

$$CPI_A = 0.2 \times 2 + 0.80 \times 1 = 1.2$$

则CPU_A性能为

$$\text{总CPU时间}_A = IC_A \times 1.2 \times \text{时钟周期}_A$$

根据假设，有

$$\text{时钟周期}_B = 1.25 \times \text{时钟周期}_A$$

在CPU_B中没有独立的比较指令，所以CPU_B的程序量为CPU_A的80%，分支指令的比例为

$$20\%/80\% = 25\%$$

这些分支指令占用2个时钟周期，而剩下的75%的指令占用1个时钟周期，因此

$$CPI_B = 0.25 \times 2 + 0.75 \times 1 = 1.25$$

因为CPU_B不执行比较，故

$$IC_B = 0.8 \times IC_A$$

因此CPU_B性能为

$$\begin{aligned} \text{总CPU时间}_B &= IC_B \times CPI_B \times \text{时钟周期}_B \\ &= 0.8 \times IC_A \times 1.25 \times (1.25 \times \text{时钟周期}_A) \\ &= 1.25 \times IC_A \times \text{时钟周期}_A \end{aligned}$$

在这些假设之下，尽管CPU_B执行指令条数较少，CPU_A因为有着更短的时钟周期，所以比CPU_B快。

如果CPU_B的时钟周期时间仅仅是CPU_A的1.1倍，则

$$\text{时钟周期}_B = 1.10 \times \text{时钟周期}_A$$

CPU_B的性能为

$$\begin{aligned}\text{总CPU时间}_B &= IC_B \times CPI_B \times \text{时钟周期}_B \\ &= 0.8 \times IC_A \times 1.25 \times (1.10 \times \text{时钟周期}_A) \\ &= 1.10 \times IC_A \times \text{时钟周期}_A\end{aligned}$$

因此CPU_B由于执行更少指令条数，比CPU_A运行更快。

4. 程序的局部性原理

程序执行时所访问的存储器地址分布不是随机的，而是相对地簇聚。

➤ 常用的一个经验规则

程序执行时间的90%都是在执行程序中10%的代码。

➤ 程序的时间局部性

程序即将用到的信息很可能就是目前正在使用的信息。

➤ 程序的空间局部性

程序即将用到的信息很可能与目前正在使用的信息在空间上相邻或者临近。

1.3.2 计算机系统的性能评测

1. 执行时间和吞吐率

如何评测一台计算机的性能，与测试者看问题的角度有关。

- 用户关心的是：单个程序的执行时间（执行单个程序所花的时间很少）
- 数据处理中心的管理人员关心的是：吞吐率（在单位时间里能够完成任务很多）

假设两台计算机为X和Y，X比Y快的意思是：

对于给定任务，X的执行时间比Y的执行时间少。

X的性能是Y的 n 倍，即

$$\frac{\text{执行时间Y}}{\text{执行时间X}} = n$$

而执行时间与性能成反比，即

$$n = \frac{\text{执行时间Y}}{\text{执行时间X}} = \frac{\frac{1}{\text{性能Y}}}{\frac{1}{\text{性能X}}} = \frac{\text{性能X}}{\text{性能Y}}$$

➤ 执行时间可以有多种定义：

- 计算机完成某一任务所花费的全部时间，包括磁盘访问、存储器访问、输入/输出、操作系统开销等。
- **CPU时间**：CPU执行所给定的程序所花费的时间，不包含I/O等待时间以及运行其他程序的时间。
 - **用户CPU时间**：用户程序所耗费的CPU时间。
 - **系统CPU时间**：用户程序运行期间操作系统耗费的CPU时间。

2. 基准测试程序

- 用于测试和比较性能的基准测试程序的最佳选择是**真实应用程序**。

（例如编译器）

- 以前常采用简化了的程序，例如：
 - **核心测试程序**：从真实程序中选出的关键代码段构成的小程序。
 - **小测试程序**：简单的只有几十行的小程序。
 - **合成的测试程序**：人工合成出来的程序。

Whetstone与Dhrystone是最流行的合成测试程序。

从测试性能的角度来看，上述测试程序就不可信了。

原因：

- 这些程序比较小，具有片面性；
 - 系统结构设计者和编译器的设计者可以“合谋”把他们的计算机面向这些测试程序进行优化设计，使得该计算机显得性能更高。
- 性能测试的结果除了和采用什么测试程序有关以外，还和在什么条件下进行测试有关。
- 基准测试程序设计者对制造商的要求
- 采用同一种编译器；
 - 对同一种语言的程序都采用相同的一组编译标志。

- **一个问题：**是否允许修改测试程序的源程序
三种不同的处理方法：
 - 不允许修改。
 - 允许修改，但因测试程序很复杂或者很大，几乎是无法修改。
 - 允许修改，只要保证最后输出的结果相同。
- **基准测试程序套件：**由各种不同的真实应用程序构成。
(能比较全面地反映计算机在各个方面的处理性能)
- **SPEC系列：**最成功和最常见的测试程序套件
(美国的标准性能评估公司开发)

- 台式计算机的基准测试程序套件可以分为两大类：
处理器性能测试程序，图形性能测试程序
- SPEC89：用于测试处理器性能。10个程序（4个整数程序，6个浮点程序）
- 演化出了4个版本
 - SPEC92：20个程序
 - SPEC95：18个程序
 - SPEC2000：26个程序
 - SPEC CPU2006：29个程序
- SPEC CPU2006

整数程序12个（CINT2006）

9个是用C写的，3个是用C++写的

浮点程序17个（CFP2006）

6个是用FORTRAN写的，4个是用C++写的，3个是用C写的，4个是用C和FORTRAN混合编写的。

➤ SPEC测试程序套件中的其他一系列测试程序组件

- ❑ **SPECSFS**: 用于NFS（网络文件系统）文件服务器的测试程序。它不仅测试处理器的性能，而且测试I/O系统的性能。它重点测试吞吐率。
- ❑ **SPECWeb**: Web服务器测试程序。

- SPECviewperf: 用于测试图形系统支持OpenGL库的性能。
- SPECcapc: 用于测试图形密集型应用的性能。
- 事务处理 (TP) 性能测试基准程序: 用于测试计算机在事务处理方面的能力, 包括数据库访问和更新等。
 - 20世纪80年代中期, 一些工程师成立了称为TPC的独立组织。目的是开发用于TP性能测试的真实而又公平的基准程序。
 - 先后发布了多个版本:
TPC-A、TPC-C、TPC-H、TPC-W、TPC-App等
(主要是用于测试服务器的性能)

➤ 用于测试基于Microsoft公司的Windows系列操作系统平台的测试套件

- **PCMark04:** 中央处理器测试组、内存测试组、图形芯片测试组、硬盘测试组等。
- **Business Winstone 2004:** 主要用于测试计算机系统商业应用的综合性能。
- **Multimedia Content Creation Winstone 2004:** 主要用于测试计算机系统多媒体应用的综合性能。
- **SiSoft Sandra Pro 2004:** 一套功能强大的系统分析评比工具，拥有超过30种以上的分析与测试模块。
主要包括：CPU、存储器、I/O接口、I/O设备、主板等。

3. 性能比较

两个程序在A、B、C三台计算机上的执行时间

	A机	B机	C机	$W(1)$	$W(2)$	$W(3)$
程序1	1.00	10.00	20.00	0.50	0.909	0.999
程序2	1000.00	10.00	20.00	0.50	0.091	0.001
加权算术 平均值 $A_m(1)$	500.50	10.00	20.00			
加权算术 平均值 $A_m(2)$	91.91	10.00	20.00			
加权算术 平均值 $A_m(3)$	2.00	10.00	20.00			

如何比较这3台计算机的性能呢？

从该表可以得出：

执行程序1：

- A机的速度是B机的10倍
- A机的速度是C机的20倍
- B机的速度是C机的2倍

执行程序2：

- B机的速度是A机的100倍
- C机的速度是A机的50倍
- B机的速度是C机的2倍

- **总执行时间：**计算机执行所有测试程序的总时间
 - B机执行程序1和程序2的速度是A机的50.05倍
 - C机执行程序1和程序2的速度是A机的24.02倍
 - B机执行程序1和程序2的速度是C机的2倍
- **平均执行时间：**各测试程序执行时间的算术平均值

$$S_m = \frac{1}{n} \sum_{i=1}^n T_i$$

其中， T_i ：第*i*个测试程序的执行时间

n ：测试程序组中程序的个数

- **加权执行时间：**各测试程序执行时间的加权平均值

$$A_m = \sum_{i=1}^n W_i \cdot T_i$$

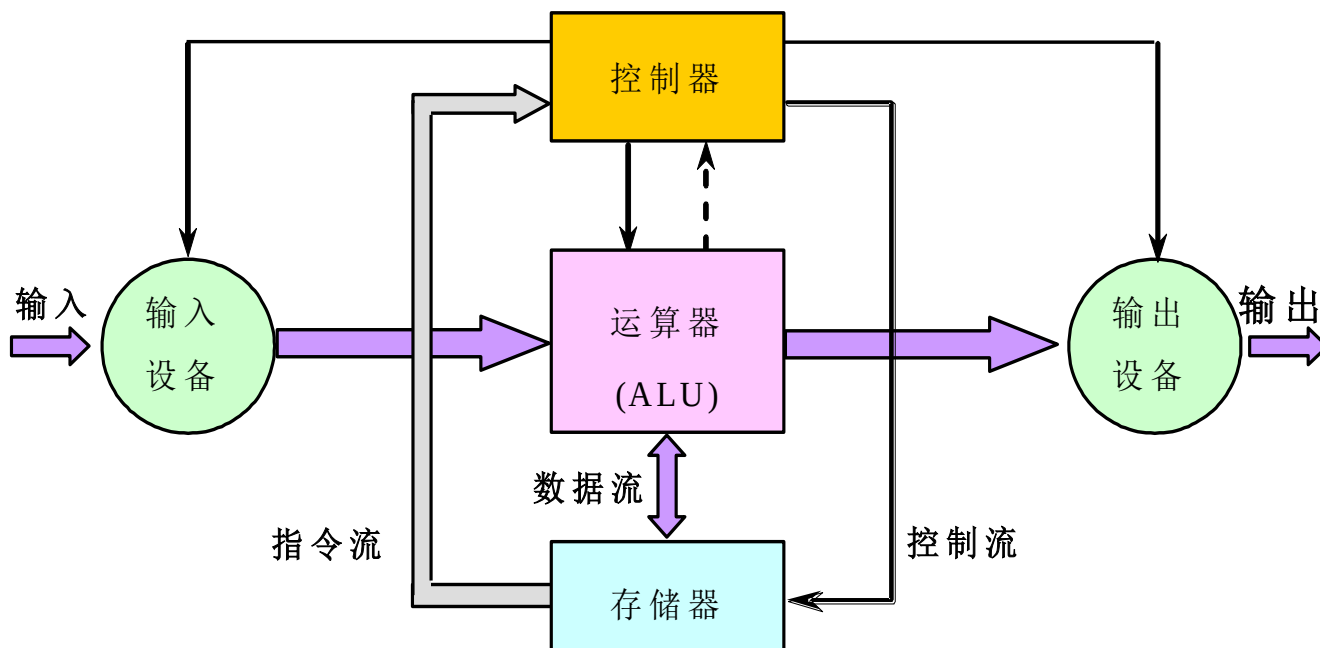
其中， W_i ：第*i*个测试程序在测试程序组中所占的比重

$$\sum_{i=1}^n W_i = 1$$

T_i ：该程序的执行时间

1.4 计算机系统结构的发展

1.4.1 冯 诺依曼结构



存储程序计算机的结构

1. 存储程序原理的基本点：指令驱动

程序预先存放在计算机存储器中，计算机一旦启动，就能按照程序指定的逻辑顺序执行这些程序，自动完成由程序所描述的处理工作。

2. 冯 诺依曼结构的主要特点

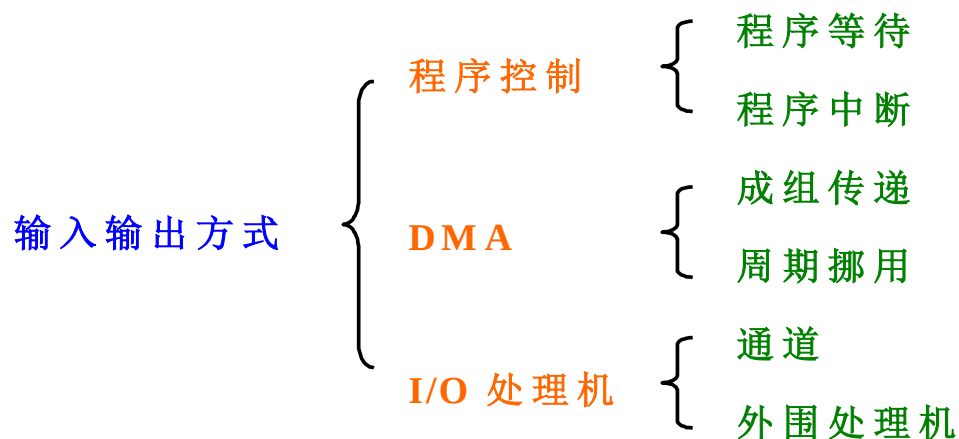
- 以运算器为中心。
- 在存储器中，指令和数据同等对待。

指令和数据一样可以进行运算，即由指令组成的程序是可以修改的。
- 存储器是按地址访问、按顺序线性编址的一维结构，每个单元的位数是固定的。

- 指令的执行是顺序的。
 - 一般是按照指令在存储器中存放的顺序执行。
 - 程序的分支由转移指令实现。
 - 由指令计数器PC指明当前正在执行的指令在存储器中的地址。
- 指令由操作码和地址码组成。
- 指令和数据均以二进制编码表示，采用二进制运算。

3. 对系统结构进行的改进

➤ 输入/输出方式的改进



➤ 采用并行处理技术

- 如何挖掘传统机器中的并行性？
- 在不同的级别采用并行技术。

例如，微操作级、指令级、线程级、进程级、任务级等。

- 存储器组织结构的发展
 - 相联存储器与相联处理机
 - 通用寄存器组
 - 高速缓冲存储器Cache

- 指令集的发展

两个发展方向：

- 复杂指令集计算机（CISC）
- 精简指令集计算机（RISC）

1.4.2 软件对系统结构的影响

- **软件的可移植性**：一个软件可以不经修改或者只需少量修改就可以由一台计算机移植到另一台计算机上正确地运行。差别只是执行时间的不同。

我们称这两台计算机是**软件兼容**的。

- **实现可移植性的常用方法**

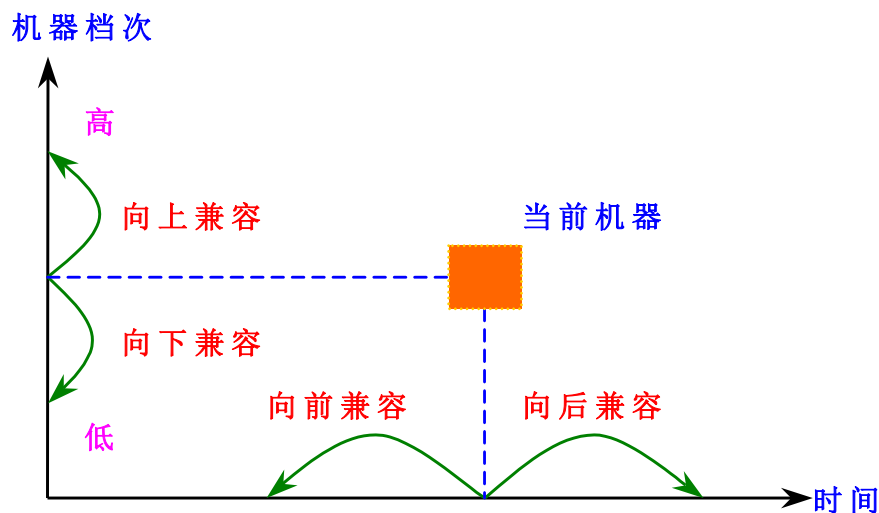
采用系列机，模拟与仿真，统一高级语言。

1. 系列机

由同一厂家生产的具有相同的系统结构，但具有不同组成和实现的一系列不同型号的机器。

- 较好地解决软件开发要求系统结构相对稳定与器件、硬件技术迅速发展的矛盾。

➤ 软件兼容



- **向上（下）兼容：**按某档机器编制的程序，不加修改就能运行于比它高（低）档的机器。
- **向前（后）兼容：**按某个时期投入市场的某种型号机器编制的程序，不加修改地就能运行于在它之前（后）投入市场的机器。

向后兼容是系列机的根本特征。

- **兼容机：**由不同公司厂家生产的具有相同系统结构的计算机。

2. 模拟和仿真

- 使软件能在具有不同系统结构的机器之间相互移植。
 - 在一种系统结构上实现另一种系统结构。
 - 从指令集的角度来看，就是要在一种机器上实现另一种机器的指令集。
- **模拟**：用软件的方法在一台现有的机器（称为**宿主机**）上实现另一台机器（称为**虚拟机**）的指令集。
 - 通常用解释的方法来实现。
 - 运行速度较慢，性能较差。

- **仿真**：用一台现有机器（**宿主机**）上的微程序去解释实现另一台机器（**目标机**）的指令集。
 - 运行速度比模拟方法的快
 - 仿真只能在系统结构差距不大的机器之间使用

3. 统一高级语言

- 实现软件移植的一种理想的方法
- 较难实现

1.4.3 器件发展对系统结构的影响

1. 摩尔定律

集成电路芯片上所集成的晶体管数目每隔18个月就翻一番。

2. 计算机的分代主要以器件作为划分标准。

- 它们在器件、系统结构和软件技术等方面都有各自的特征。
- **SMP**：对称式共享存储器多处理机
- MPP**：大规模并行处理机

分代	器件特征	结构特征	软件特征	典型实例
第一代 (1945—1954年)	电子管和继电器	存储程序计算机 程序控制I/O	机器语言 汇编语言	普林斯顿ISA, ENIAC, IBM 701
第二代 (1955—1964年)	晶体管、磁芯 印刷电路	浮点数据表示 寻址技术 中断、I/O处理机	高级语言和编译 批处理监控系统	Univac LAPC, CDC 1604, IBM 7030
第三代 (1965—1974年)	SSI和MSI 多层印刷电路 微程序	流水线、Cache 先行处理 系列机	多道程序 分时操作系统	IBM 360/370, CDC 6600/7600, DEC PDP-8
第四代 (1975—1990年)	LSI和VLSI 半导体存储器	向量处理 分布式存储器	并行与分布处理	Cray-1, IBM 3090, DEC VAX 9000, Convax-1
第五代 (1991年—)	高性能微处理器 高密度电路	超标量、超流水 SMP、MP、MPP 机群	大规模、可扩展 并行与分布处理	SGI Cray T3E, IBM SP2, DEC AlphaServer 8400

1.4.4 应用对系统结构的影响

1. 不同的应用对计算机系统结构的设计提出了不同的要求
2. 应用需求是促使计算机系统结构发展的最根本的动力
3. 一些特殊领域：需要高性能的系统结构
 - 高结构化的数值计算
气象模型、流体动力学、有限元分析
 - 非结构化的数值计算
蒙特卡洛模拟、稀疏矩阵
 - 实时多因素问题
语音识别、图像处理、计算机视觉

- 大存储容量和输入输出密集的问题
数据库系统、事务处理系统
- 图形学和设计问题
计算机辅助设计
- 人工智能
面向知识的系统、推理系统等

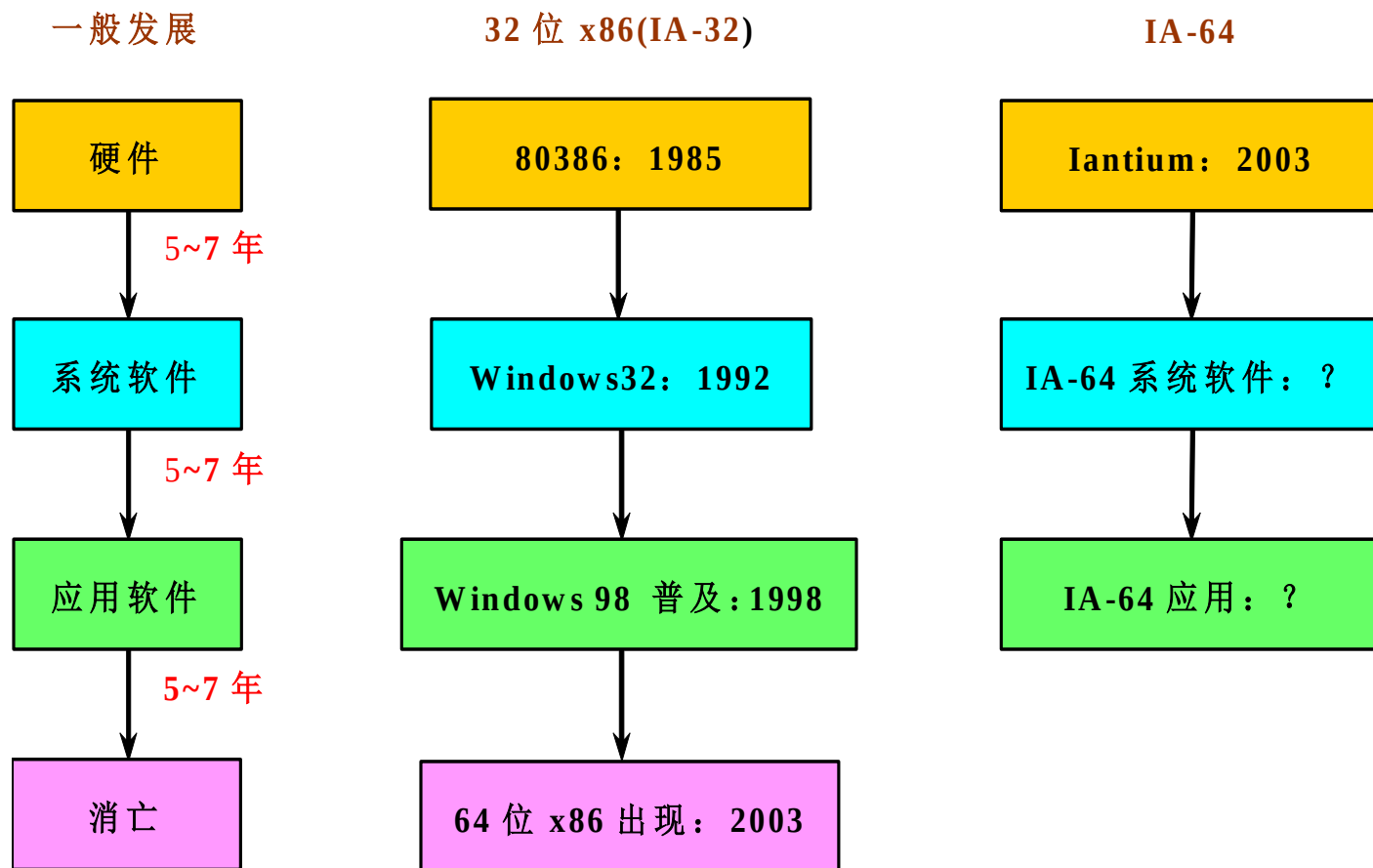
4. 计算机性能随时间下移

1.4.5 系统结构的生命周期

1. 系统结构的生命周期：从诞生、发展、成熟到消亡

- 从硬件成熟到系统软件成熟大约需要5~7年的时间
- 从系统软件成熟到应用软件成熟，大约也需要5~7年时间。
- 再过5~7年的时间，这种系统结构就不会作为主流系统结构存在了。

2. Intel的x86系列微处理器中32位系统结构的发展



1.5 计算机系统结构中并行性的发展

1.5.1 并行性的概念

1. **并行性**：计算机系统在同一时刻或者同一时间间隔内进行多种运算或操作。

只要在时间上相互重叠，就存在并行性。

- **同时性**：两个或两个以上的事件在同一时刻发生。
- **并发性**：两个或两个以上的事件在同一时间间隔内发生。

2. 从处理数据的角度来看，并行性等级从低到高可分为：

- **字串位串**：每次只对一个字的一位进行处理。
最基本的串行处理方式，不存在并行性。
- **字串位并**：同时对一个字的全部位进行处理，不同字之间是串行的。
开始出现并行性。
- **字并位串**：同时对许多字的同一位（称为**位片**）进行处理。
具有较高的并行性。
- **全并行**：同时对许多字的全部位或部分位进行处理。
最高一级的并行。

3. 从执行程序的角度来看，并行性等级从低到高可分为：

- **指令内部并行：**单条指令中各微操作之间的并行。
- **指令级并行：**并行执行两条或两条以上的指令。
- **线程级并行：**并行执行两个或两个以上的线程。

通常是以一个进程内派生的多个线程为调度单位。

- **任务级或过程级并行：**并行执行两个或两个以上的过程或任务（程序段）

以子程序或进程为调度单元。

- **作业或程序级并行：**并行执行两个或两个以上的作业或程序。

1.5.2 提高并行性的技术途径

三种途径：

1. 时间重叠

引入时间因素，让多个处理过程在时间上相互错开，轮流重叠地使用同一套硬件设备的各个部分，以加快硬件周转而赢得速度。

2. 资源重复

引入空间因素，以数量取胜。通过重复设置硬件资源，大幅度地提高计算机系统的性能。

3. 资源共享

这是一种软件方法，它使多个任务按一定时间顺序轮流使用同一套硬件设备。

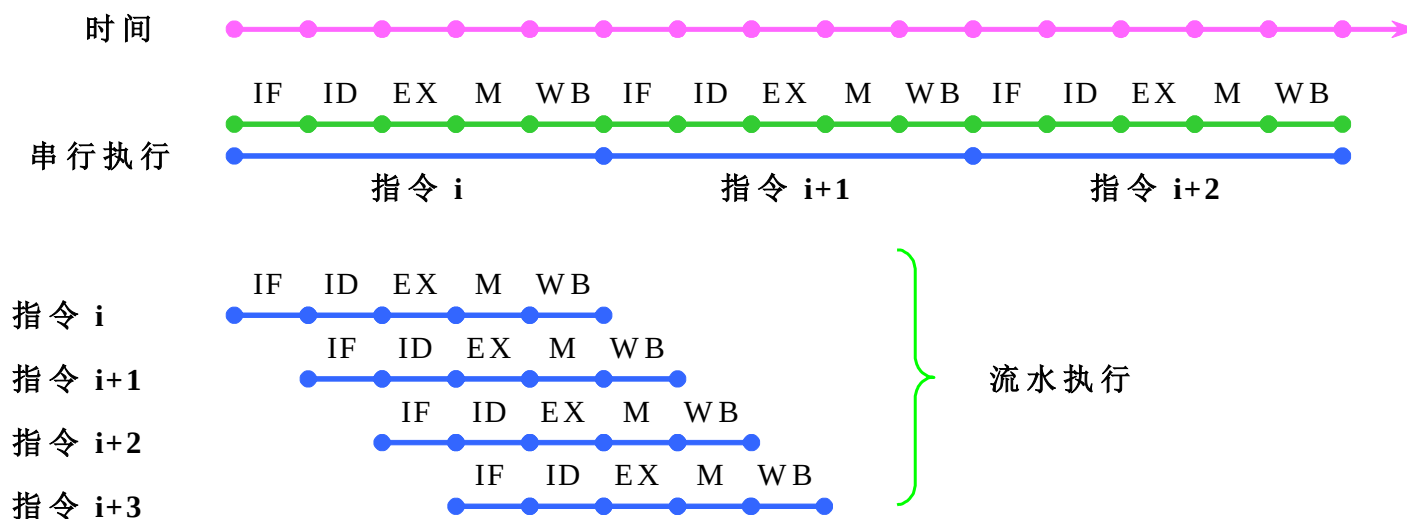
1.5.3 单机系统中并行性的发展

1. 在发展高性能单处理机过程中，起主导作用的是时间重叠原理。

实现时间重叠的基础：部件功能专用化

- 把一件工作按功能分割为若干相互联系的部分；
- 把每一部分指定给专门的部件完成；
- 按时间重叠原理把各部分的执行过程在时间上重叠起来，使所有部件依次分工完成一组同样的工作。

例如： 对于解释指令的5个过程，就分别需要5个专用的部件：取指令部件 (IF)、指令译码部件 (ID)、指令执行部件 (EX)、访问存储器部件 (M) 和写结果部件 (WB)。



2. 在单处理机中，资源重复原理的运用也已经十分普遍。

➤ 多体存储器

➤ 多操作部件

- 通用部件被分解成若干个专用部件，如加法部件、乘法部件、除法部件、逻辑运算部件等，而且同一种部件也可以重复设置多个。
- 只要指令所需的操作部件空闲，就可以开始执行这条指令（如果操作数已准备好的话）。
- 这实现了指令级并行。

➤ 阵列处理机（并行处理机）

更进一步，设置许多相同的处理单元，让它们在同一个控制器的指挥下，按照同一条指令的要求，对向量或数组的各元素同时进行同一操作，就形成了阵列处理机。

3. 在单处理机中，资源共享的概念实质上是用单处理机模拟多处理机的功能，形成所谓虚拟机的概念。

□ 分时系统

1.5.4 多机系统中并行性的发展

1. 多机系统遵循时间重叠、资源重复、资源共享原理，发展为3种不同的多处理机：

异构型多处理机、同构型多处理机、分布式系统

2. 耦合度

反映多机系统中各机器之间物理连接的紧密程度和交互作用能力的强弱。

- 紧密耦合系统（直接耦合系统）：在这种系统中，计算机之间的物理连接的频带较高，一般是

通过总线或高速开关互连，可以共享主存。

- **松散耦合系统（间接耦合系统）**：一般是通过通道或通信线路实现计算机之间的互连，可以共享外存设备（磁盘、磁带等）。机器之间的相互作用是在文件或数据集一级上进行的。

表现为两种形式：

- 多台计算机和共享的外存设备连接，不同机器之间实现功能上的分工（功能专用化），机器处理的结果以文件或数据集的形式送到共享外存设备，供其他机器继续处理。
- 计算机网络，通过通信线路连接，实现更大范围的资源共享。

3. 功能专用化（实现时间重叠）

- 专用外围处理机

例如，输入/输出功能的分离。

- 专用处理机

如数组运算、高级语言翻译、数据库管理等，分离出来。

- 异构型多处理机系统

由多个不同类型、至少担负不同功能的处理机组成，它们按照作业要求的顺序，利用时间重叠原理，依次对它们的多个任务进行加工，各自完成规定的功能动作。

4. 机间互连

- 容错系统
- 可重构系统

对计算机之间互连网络的性能提出了更高的要求。
高带宽、低延迟、低开销的机间互连网络是高效实现程序或任务一级并行处理的前提条件。

- 同构型多处理机系统

由多个同类型或至少担负同等功能的处理机组成，
它们同时处理同一作业中能并行执行的多个任务。

附1：影响计算机系统的成本和价格因素

1. 计算机系统的成本和价格

组装一台较高档配置PC的硬件各部件价格分布

- 从总体看，价格变化的趋势是不断下降的。
- 计算机的价格是与成本紧密相关的。

一台组装PC及其各个部件的价格分布

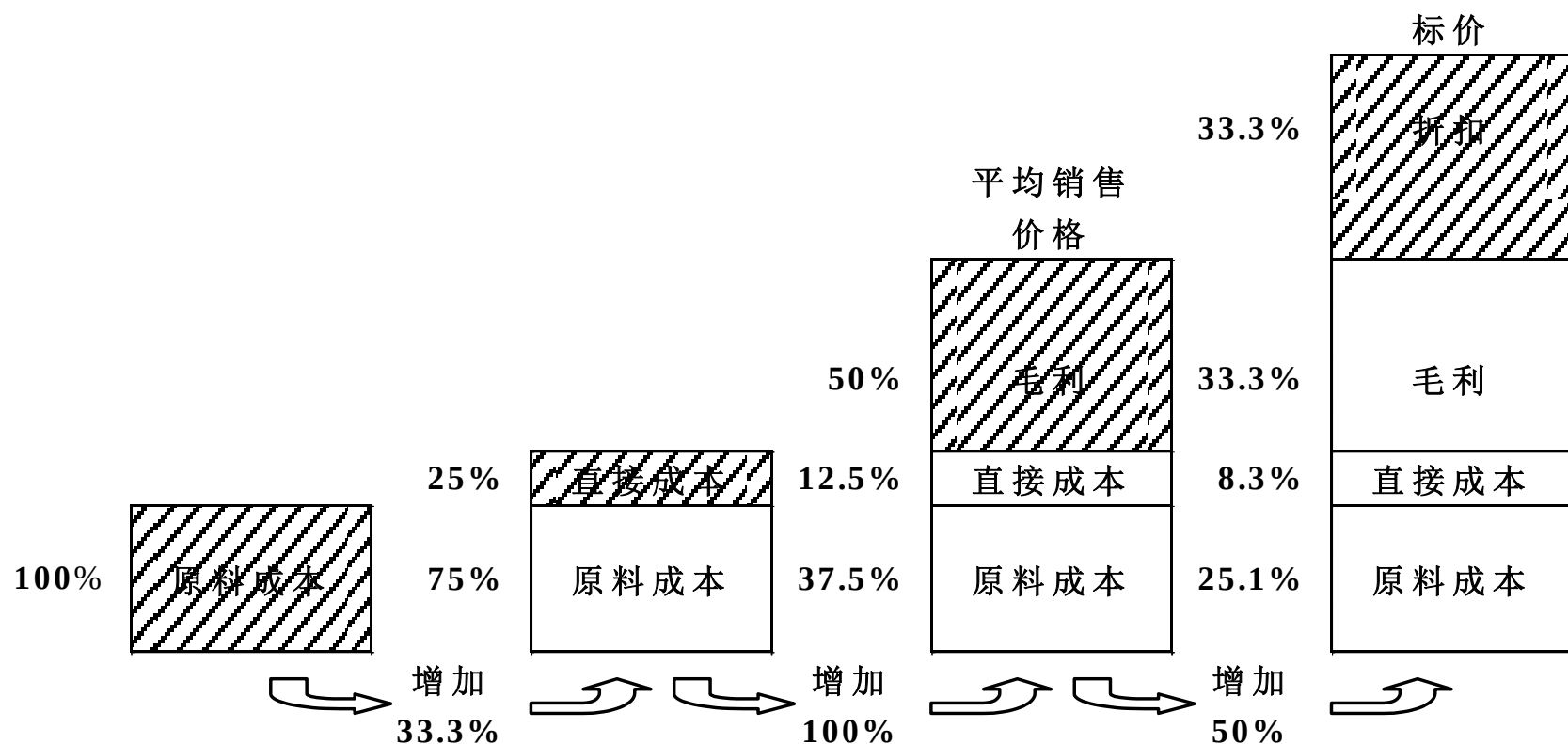
配件	型号	价格（¥）	总价格中的比例
处理器	Pentium 4 2.6C GHz	1400	17%
主板（含基本I/O）	微星865PE-Neo2-PFS	980	12%
存储器	Kingston 256M DDR400（两条）	600	7%
硬盘	三星SP80GB 2MB缓冲	680	8%
显示卡	Unika GeForce FX5900FXc 128MB DDR	1699	20%
显示器	Magvision PR700M2	1399	17%
光驱	MIDA 48× Combo	390	5%
键盘和鼠标	BenQ海湾键盘+微软IE4.0	498	6%
音箱	漫步者R301T II	260	3%
其他配件	机箱、电源、软驱、CPU风扇等	420	5%
合计：		8326	

2. 价格与成本是不同的概念

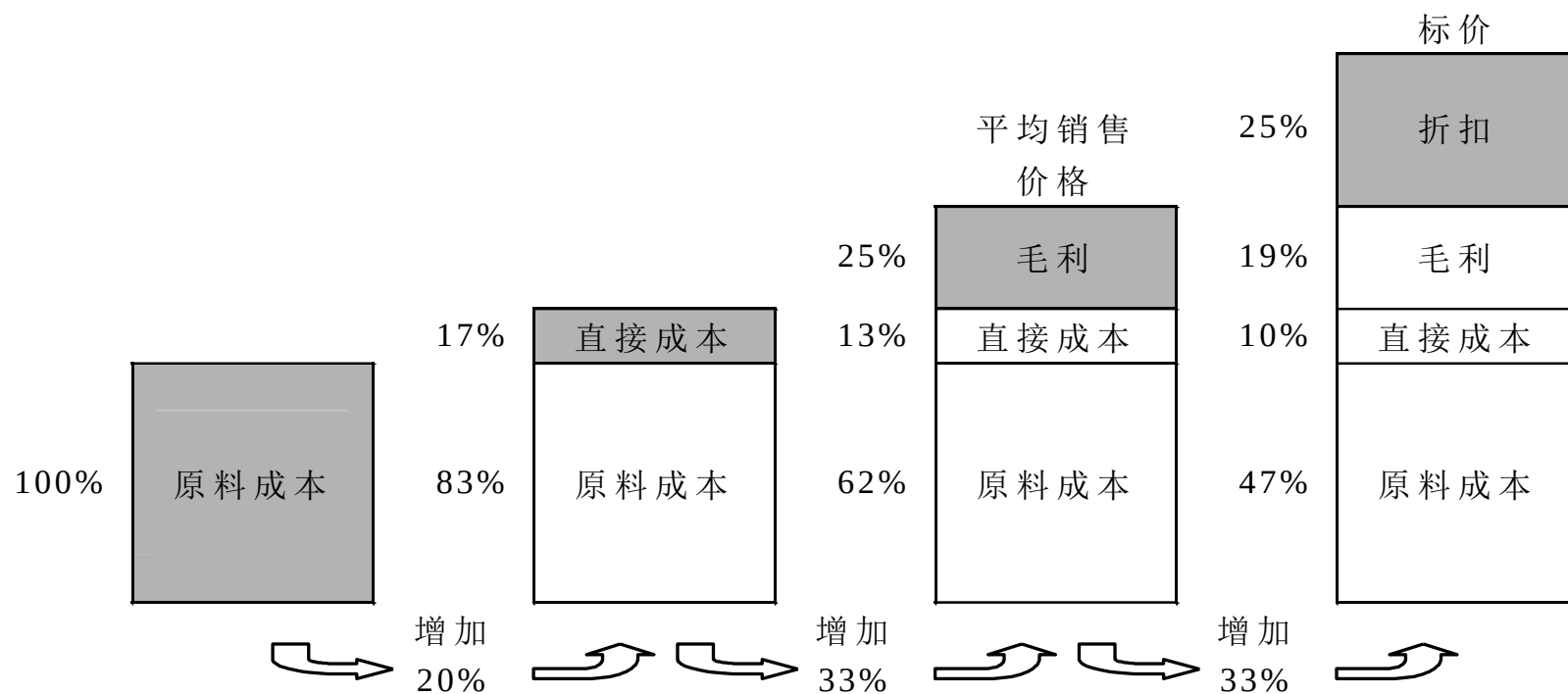
商品的标价(价格)由这样一些因素构成：

- 原料成本：一件产品中所有部件的采购成本总和。
- 直接成本：与一件产品生产直接相关的成本。
- 毛利，主要包括：
 - 公司的研发费用
 - 市场建立费用
 - 销售费用
 - 生产设备维护费用
 - 房租
 - 贷款利息
 - 税后利润和所得税
- 平均销售价格与折扣

3. 工作站的成本和价格



4. PC的成本和价格



5. 影响计算机系统成本的主要因素

➤ 时间

（对成本产生最直接影响）

即使实现技术没有变动，计算机系统的制造成本也会不断下降。

➤ 产量

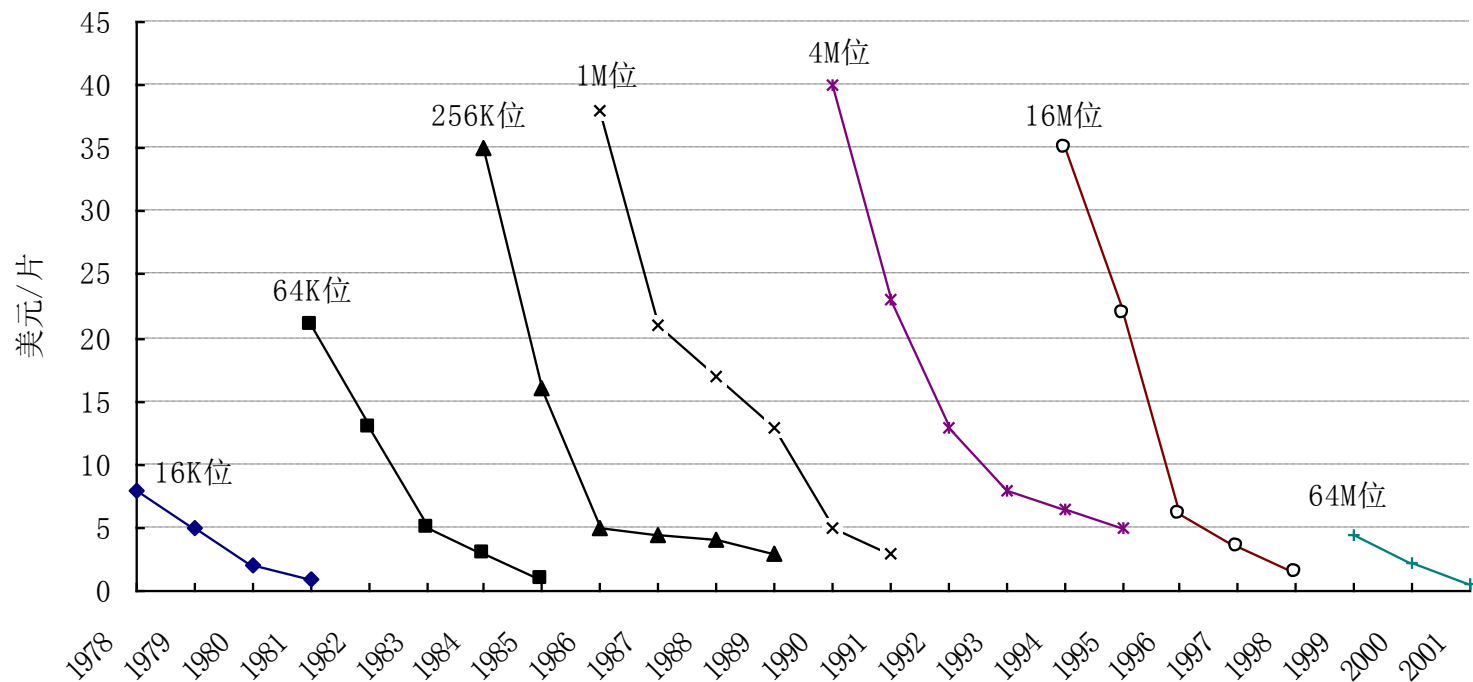
- 产量的增加会加速工艺的穩定；
- 产量增加提高了生产效率，降低了成本；
- 产量增加降低每台单机必须加入的开发费用，从而使得单机成本下降。

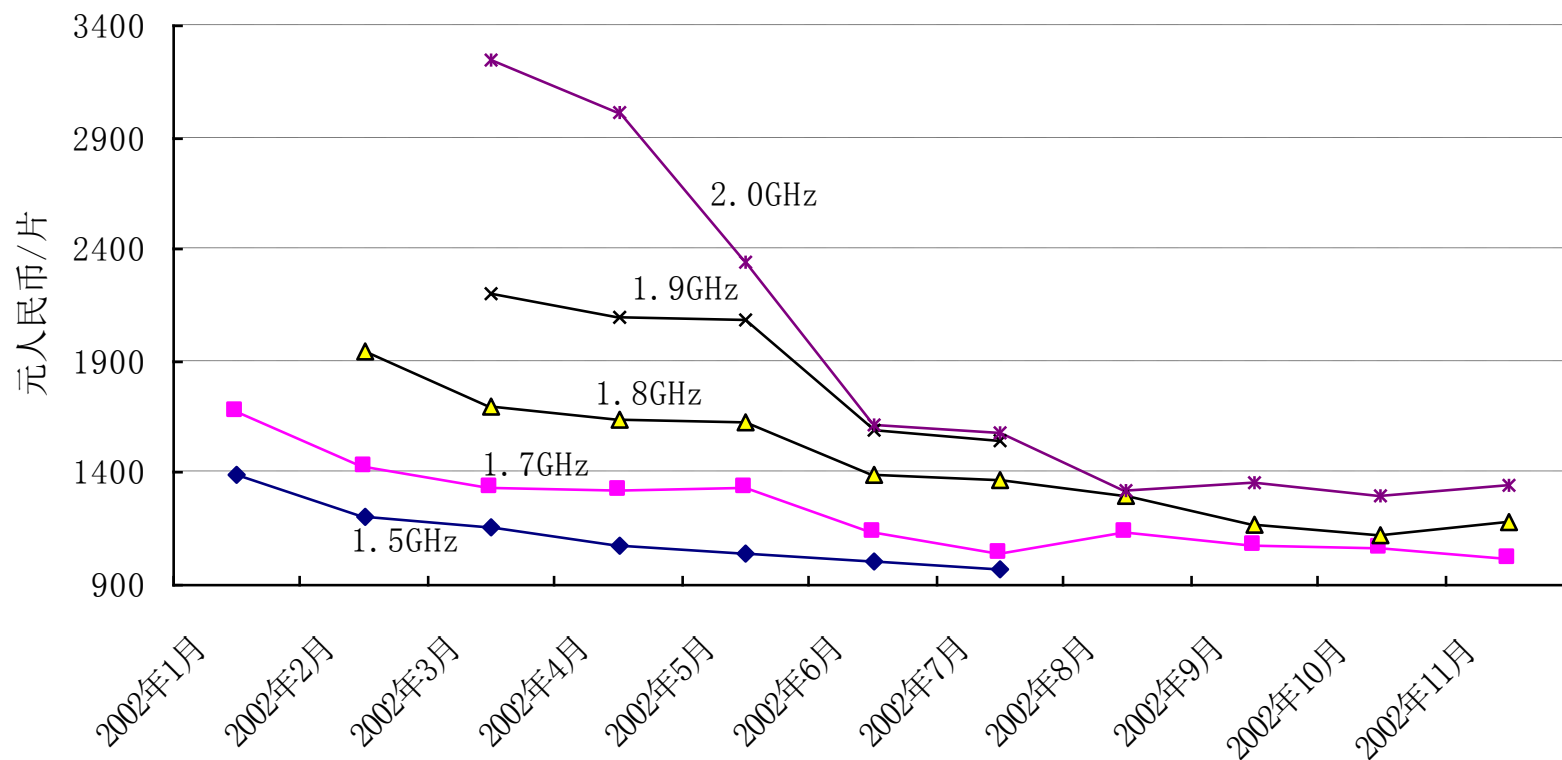
➤ 商品化

（更重要的是它影响产品的价格）

➤ 价格随时间下降

- 存储器价格变化的学习曲线
- **2002年Intel的一些Pentium 4处理器零售**
- 价格变化的学习曲线

**DRAM价格的学习曲线**



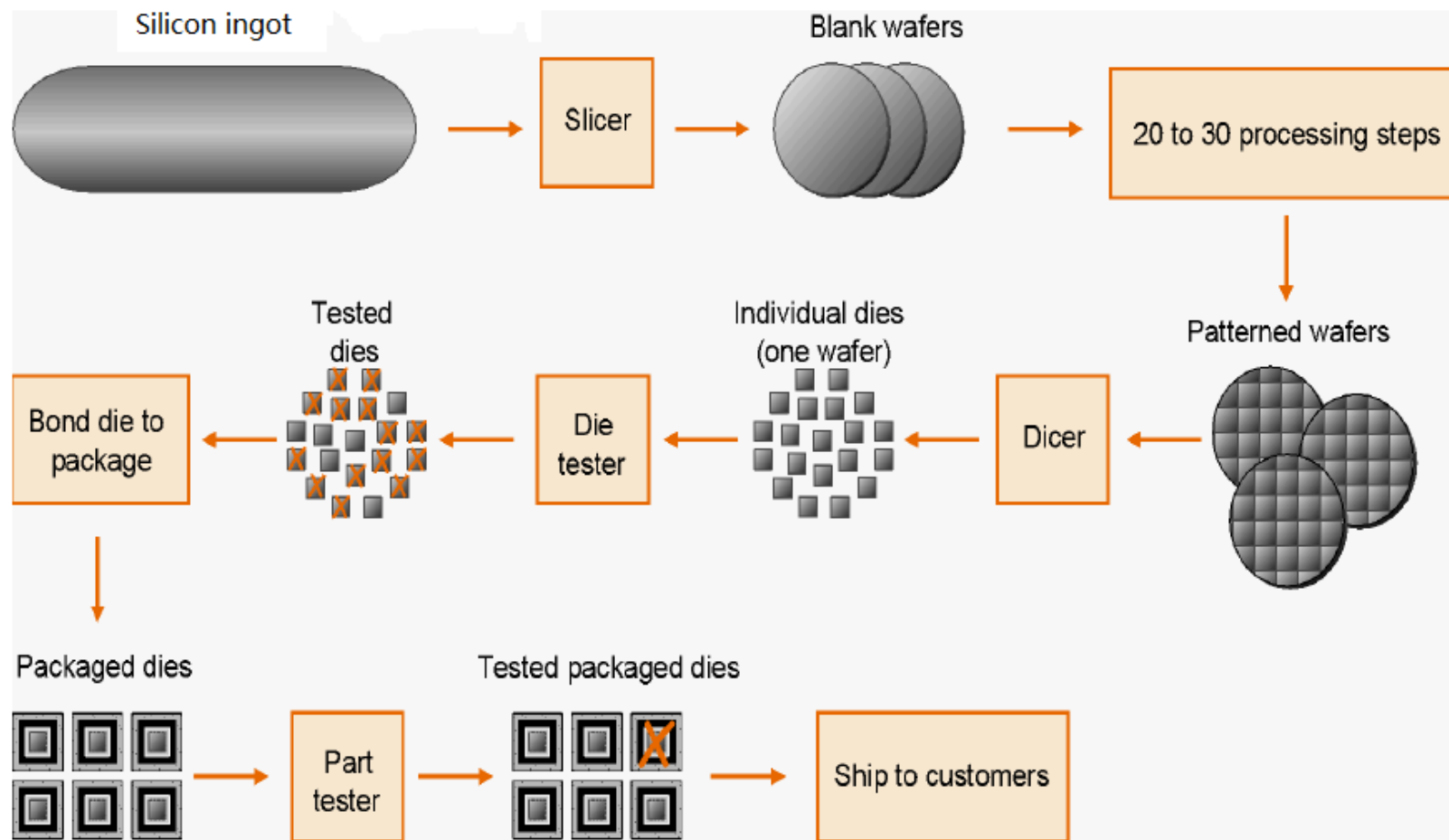
Intel公司部分型号Pentium 4处理器零售价格的学习曲线

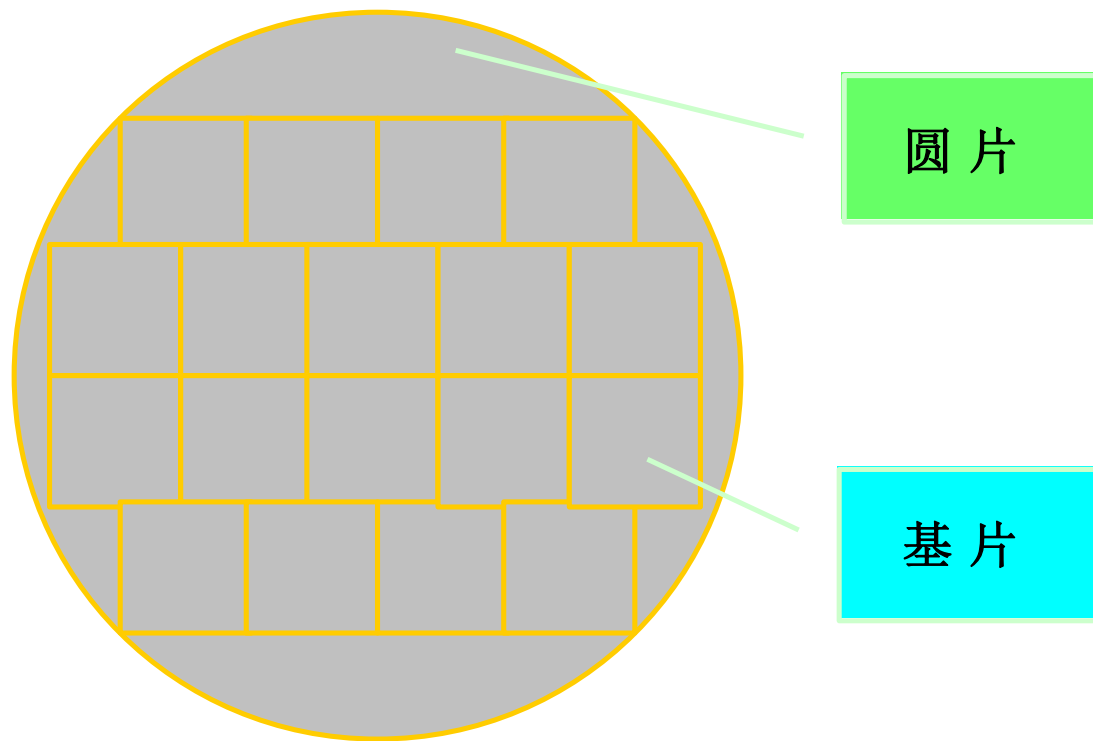
6. 集成电路的成本

集成电路的基本制造工艺：

- 首先生产圆片；
- 在圆片上制造出大量电路单元；
- 圆片经过测试后按照制造的电路单元被切割成基片；
- 基片在外壳中封装好以后就是集成电路成品。

附2：芯片制作流程





集成电路的圆片和基片

$$\text{集成电路成本} = \frac{\text{基片成本} + \text{基片测试成本} + \text{封装成本}}{\text{最终成品率}}$$

$$\text{基片成本} = \frac{\text{圆片成本}}{\text{每块圆片上的基片数} \times \text{基片成品率}}$$

$$\text{每块圆片上的基片数} = \frac{\text{圆片面积}}{\text{基片面积}} = \frac{\pi \times \text{圆片直径}}{\sqrt{2} \times \text{基片面积}}$$

$$\text{基片成品率} = \text{园片成品率} \times \left(1 + \frac{\text{疵点密度} \times \text{基片面积}}{\alpha}\right)^{-\alpha}$$

$$\text{基片成本} = \frac{\text{圆片成本}}{\text{每块圆片上的基片数} \times \text{基片成品率}}$$

$$= \frac{\text{圆片成本}}{\left(\frac{\text{圆片面积}}{\text{基片面积}} - \frac{\pi \times \text{圆片直径}}{\sqrt{2 \times \text{基片面积}}} \right) \times \text{园片成品率} \times \left(1 + \frac{\text{疵点密度} \times \text{基片面积}}{\alpha} \right)^{-\alpha}}$$

- 园片成本、园片成品率、 α 值及疵点密度等由生产工艺决定；
- 基片面积由设计人员控制。